RKNN2 Supported Operators and Constraints Reference

Document ID: RK-YH-YF-418

Release Version: v2.5.0

Date: 2026-09-29

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip.

All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. © 2026 Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

Address: No.18 Building, A District, Software Boulevard, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

RKNN2 Supported Operators and Constraints Reference

Intended Audience

This document (guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
Earlier versions	NPU Team	—	Details
V2.0.0-beta	NPU Team	2024-03-22	Details
V2.1.0	NPU Team	2024-08-07	Details
V2.2.0	NPU Team	2024-09-04	Details
V2.3.0	NPU Team	2024-11-04	Details
V2.3.2	NPU Team	2025-03-28	Details
V2.4.0	NPU Team	2026-01-15	Details
V2.5.0	NPU Team	2026-09-29	Details

Table of Contents

- RKNN2 Supported Operators and Constraints Reference
  - Revision History
    - V2.5.0
    - V2.4.0
    - V2.3.2
    - V2.3.0
    - V2.2.0
    - V2.1.0
    - V2.0.0-beta
  - NPU OP Specifications
    - RK3566/RK3568
      - Add
      - AddRelu
      - BatchNormalization
      - Concat
      - Convolution
      - Depthwise Convolution
      - ConvolutionRelu
      - ConvolutionClip
      - ConvolutionPRelu/LeakyRelu
      - ConvolutionAdd
      - ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

LSTM

Max

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

Relu

- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- exSwooshR
- exSwooshL
- Where
- exGlu
- exMatMul
- exNorm
- exWindow

RK3588

- Add
- AddRelu
- BatchNormalization
- Concat
- Convolution
- Depthwise Convolution
- ConvolutionRelu
- ConvolutionClip
- ConvolutionPRelu/LeakyRelu
- ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

Hardmax

LSTM

Max

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

- Pow
- Relu
- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Softmax
- exSoftmax13
- exSoftmaxMask
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- exSwooshR
- exSwooshL
- Where
- exConvStreaming
- exConvStreamingPRelu/LeakyRelu
- exGlu
- exMatMul
- exNorm
- exSDPAttention
- exWindow

RK3562

- Add
- AddRelu

BatchNormalization

Concat

Convolution

Depthwise Convolution

ConvolutionRelu

ConvolutionClip

ConvolutionPRelu/LeakyRelu

ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvolutionAddRelu

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

ConvTranposeAdd

ConvTranposeSigmoid

ConvTranposeTanh

ConvTranposeSoftplus

ConvTranposeHardSigmoid

ConvTranposeHardSwish

ConvTranposeElu

ConvTranposeSwish

ConvTranposeMish

Div

Expand

GRU

Gather

Hardmax

LSTM

Max

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

Relu

Clip/ReLU6

PRelu

LeakyRelu

Reshape

Resize

Slice

Softmax

exSoftmax13

exSoftmaxMask

Split

Sub

Tile

Transpose

Sigmoid

Tanh

Softplus

HardSigmoid

HardSwish

Elu

exSwish

exMish

exGelu

Where

exGlu

exMatMul

exNorm

exSDPAttention

exWindow

RV1103/RV1106

- Add
- AddRelu
- BatchNormalization
- Concat
- Convolution
- Depthwise Convolution
- ConvolutionRelu
- ConvolutionClip
- ConvolutionPRelu/LeakyRelu
- ConvolutionAdd
- ConvolutionSigmoid
- ConvolutionTanh
- ConvolutionSoftplus
- ConvolutionHardSigmoid
- ConvolutionHardSwish
- ConvolutionElu
- ConvolutionSwish
- ConvolutionMish
- ConvolutionAddRelu
- ConvTranspose
- ConvTranposeRelu
- ConvTranposeClip
- ConvTranposePRelu/LeakyRelu
- ConvTranposeAdd
- ConvTranposeSigmoid
- ConvTranposeTanh
- ConvTranposeSoftplus
- ConvTranposeHardSigmoid
- ConvTranposeHardSwish
- ConvTranposeElu
- ConvTranposeSwish
- ConvTranposeMish
- Div
- Expand
- Hardmax
- LSTM

Max

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

Relu

Clip/ReLU6

PRelu

LeakyRelu

Reshape

Resize

Slice

Softmax

exSoftmax13

exSoftmaxMask

Split

Sub

Tile

Transpose

Sigmoid

Tanh

Softplus

HardSigmoid

HardSwish

Elu

exSwish

exMish

exGelu

Where

exGlu

exMatMul

exSDPAttention

- exWindow

RK3576/RV1126B

- Add
- AddRelu
- BatchNormalization
- Concat
- Convolution
- Depthwise Convolution
- ConvolutionRelu
- ConvolutionClip
- ConvolutionPRelu/LeakyRelu
- ConvolutionAdd
- ConvolutionSigmoid
- ConvolutionTanh
- ConvolutionSoftplus
- ConvolutionHardSigmoid
- ConvolutionHardSwish
- ConvolutionElu
- ConvolutionSwish
- ConvolutionMish
- ConvolutionAddRelu
- ConvTranspose
- ConvTranposeRelu
- ConvTranposeClip
- ConvTranposePRelu/LeakyRelu
- ConvTranposeAdd
- ConvTranposeSigmoid
- ConvTranposeTanh
- ConvTranposeSoftplus
- ConvTranposeHardSigmoid
- ConvTranposeHardSwish
- ConvTranposeElu
- ConvTranposeSwish
- ConvTranposeMish
- Div
- Expand
- GRU

Gather

Hardmax

LSTM

Max

Greater

Min

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Pow

ReduceMax

Relu

Clip/ReLU6

PRelu

LeakyRelu

Reshape

Resize

Slice

Softmax

exSoftmax13

exSoftmaxMask

Split

Sub

Tile

Transpose

Sigmoid

Tanh

Softplus

HardSigmoid

HardSwish

Elu

exSwish

exMish

- exGelu
- Sin
- Cos
- Where
- exConvStreaming
- exConvStreamingPRelu/LeakyRelu
- exGlu
- exMatMul
- exNorm
- exSDPAttention
- exWindow

RK2118

- Add
- AddRelu
- BatchNormalization
- Concat
- Convolution
- Depthwise Convolution
- ConvolutionRelu
- ConvolutionClip
- ConvolutionPRelu/LeakyRelu
- ConvolutionAdd
- ConvolutionSigmoid
- ConvolutionTanh
- ConvolutionSoftplus
- ConvolutionHardSigmoid
- ConvolutionHardSwish
- ConvolutionElu
- ConvolutionSwish
- ConvolutionMish
- ConvTranspose
- Div
- Expand
- LSTM
- Max
- Mul
- MulRelu

- Pad
- MaxPool
- GlobalMaxPool
- AveragePool
- GlobalAveragePool
- Pow
- Relu
- Clip/ReLU6
- PRelu
- LeakyRelu
- Reshape
- Resize
- Slice
- Softmax
- exSoftmax13
- exSoftmaxMask
- Split
- Sub
- Tile
- Transpose
- Sigmoid
- Tanh
- Softplus
- HardSigmoid
- HardSwish
- Elu
- exSwish
- exMish
- exGelu
- Where
- exConvStreaming
- exConvStreamingPRelu/LeakyRelu
- exGlu
- exMatMul
- exWindow

RV1103B/RV1106B

- Add

AddRelu

BatchNormalization

Concat

Convolution

Depthwise Convolution

ConvolutionRelu

ConvolutionClip

ConvolutionPRelu/LeakyRelu

ConvolutionAdd

ConvolutionSigmoid

ConvolutionTanh

ConvolutionSoftplus

ConvolutionHardSigmoid

ConvolutionHardSwish

ConvolutionElu

ConvolutionSwish

ConvolutionMish

ConvTranspose

ConvTranposeRelu

ConvTranposeClip

ConvTranposePRelu/LeakyRelu

Div

Expand

Max

Mul

MulRelu

Pad

MaxPool

GlobalMaxPool

AveragePool

GlobalAveragePool

Relu

Clip/ReLU6

PRelu

LeakyRelu

Reshape

Resize

Slice
Softmax
exSoftmax13
exSoftmaxMask
Split
Sub
Tile
Transpose
Sigmoid
Tanh
Softplus
HardSigmoid
HardSwish
Elu
exSwish
exMish
exGelu
Where
exGlu
exMatMul
exWindow
CPU OP Specifications
Note:
Add
AveragePool
ArgMin
ArgMax
BatchNormalization
Cast
Clip
Concat
Convolution
ConvTranspose/Deconvolution
ConvTransposePad
Cos
DataConvert

DepthToSpace

Div

Equal

Exp

Flatten

Gather

Greater

GreaterOrEqual

GRU

exGRU

exHardSwish

InstanceNormalization

exLayerNorm

Less

LessOrEqual

LogSoftmax

LpNormalization

exLRN

MatMul

Max

MaxPool

MaxRoiPool

MaxUnpool

exMish

Min

Mul

Pad

Pow

exProposal

ReduceMax

ReduceMean

ReduceSum

ReduceMin

Reorg

Reshape

Resize

ReverseSequence

exRMSNorm

RoiAlign

ScatterND

Sin

Slice

Softmax

exSoftmax13

SpaceToDetph

Split

Sqrt

Squeeze

Sub

Tanh

Tile

Transpose

Upsample

Not

where

Erf

Floor

Mod

exMeanVarianceNormalization

And

GatherElements

Log

exDataConvert

SpaceToDepth

SoftmaxMask

TransposeReshape

ReshapeTranspose

ReduceL2

LayerNormalization

PassThrough

Expand

Sigmoid

Tanh

Cos

- Sin
- exSwish
- exGlu
- exGelu
- exWindow
- exNorm
- Round
- TopK
- exSwooshR
- exSwooshL
- OneHot
- Cumsum

GPU OP Specifications

- Note:
- GridSample
- GRU
- exGelu

Model I/O Limitations

- Model Input Limitations
  - RK3566/RK3568
  - RK3588
  - RV1103/RV1106
  - RK3562
  - RK3576/RV1126B
  - RK2118
  - RV1103B
- Model Output Limitations
  - RK3566/RK3568
  - RK3588
  - RV1103/RV1106
  - RK3562
  - RK3576/RV1126B
  - RK2118
  - RV1103B/RV1106B

Special Notes

- dilated_kernel_h

dilated_kernel_w

Revision History

V2.5.0

Author: NPU Team

Date: 2026-09-29

Change Description

1. Version updated to V2.5.0

V2.4.0

Author: NPU Team

Date: 2026-01-15

Change Description

1. Added GPU operator support documentation
2. Added Pow/CumSum to the CPU operator support list
3. Added ReduceMax+exConvStreaming operator documentation for the 3588 NPU implementation
4. Added ReduceMax/exConvStreaming/Sin/Cos operator documentation for the 3576 NPU implementation
5. Added ReduceMax+exConvStreaming operator documentation for the 1126B NPU implementation

V2.3.2

Author: NPU Team

Date: 2025-03-28

Change Description

1. Added RV1126B platform documentation
2. Added RV1106B platform documentation
3. Updated the Conv Dilation parameter restrictions for all platforms
4. Updated the Resize operator restrictions

V2.3.0

Author: NPU Team

Date: 2024-11-04

Change Description

1. Added documentation for the mean_var mode of exNorm on the RK3576/RK3588/RK3562/RK3568 platforms

2. MAX/MIN OPs on all platforms now support Broadcast
3. Modified the RK3576 Softmax restriction specifications

V2.2.0

Author: NPU Team

Date: 2024-09-03

Change Description

1. Modified the softmax specification restrictions on all platforms (except rk3568), and added softmax splitting
2. Added RMSNorm support for RK3576, RK3588, RK3562, and RK3568

V2.1.0

Author: NPU Team

Date: 2024-08-07

Change Description

1. Added RK2118 documentation
2. Added RV1103B documentation
3. Updated operator specifications for all platforms
4. Added RK3576 operator specifications (including Hardmax, ConvLut, etc.)
5. RK2118 exConvStreaming support
6. Added FlashAttentionV2 optimization support to the RK3562/RK3576 exSDPA operator

V2.0.0-beta

Author: NPU Team

Date: 2024-03-22

Change Description

1. Added RK3576 platform operator support specifications
2. Added hardware support specifications for exSDPAttention and exMatMul on the RK3588/RK3562/RK3576 platforms
3. Added Floor, Mod, And, GatherElements, and exMeanVarianceNormalization CPU operator support
4. Added RK2118 platform operator support specifications

NPU OP Specifications

RK3566/RK3568

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

Other Support

- Multi-core: Supported

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- epsilon: float (default value = 1e-5)
  - A real number added when dividing by the standard deviation to prevent division by 0
  - epsilon∈(0,∞]
- momentum:float
  - Sliding average parameters during training

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axisint64
  - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]

        - kernel_w∈[1,31]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.

        - Quantization support
            - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,8]
        - kernel_w∈[1,8]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

---

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAddRelu

Convolution and Add and Relu fused computation, Output=Relu(Add(Conv(Input0),Input1)). Description and specification constraints are the same as Convolution

---

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
- Quantization support
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support

        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h only supports 1,2,4,8
    - stride_w only supports 1,2,4,8
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

ConvTranposeRelu

ConvTranspose and Relu fused computation, Output=Relu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeClip

ConvTranspose and Clip fused computation, Output=Clip(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranspose and PRelu/LeakyRelu fused computation, Output=PRelu(ConvTranspose(Input)) or Output= LeakyRelu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeAdd

ConvTranspose and Add fused computation, Output=Add(ConvTranspose(Input0),Input1). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSigmoid

ConvTranspose and Sigmoid fused computation, Output=Sigmoid(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeTanh

ConvTranspose and Tanh fused computation, Output=Tanh(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSoftplus

ConvTranspose and Softplus fused computation, Output=Softplus(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranspose and HardSigmoid fused computation, Output=HardSigmoid(ConvTranspose(Input))

ConvTranposeHardSwish

ConvTranspose and HardSwish fused computation, Output=HardSwish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeElu

ConvTranspose and Elu fused computation, Output=Elu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSwish

ConvTranspose and Swish fused computation, Output=Swish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeMish

ConvTranspose and Mish fused computation, Output=Mish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

Broadcast the input tensor according to the given shape and broadcasting rules. The broadcasting rule is similar to numpy.array(input) * numpy.ones(shape): dimensions are right-aligned; two corresponding dimensions must have the same value, or one of them equals 1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- shape:T2
    - shape:[batch,channel,height,width]

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]

- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

GRU

Gated Recurrent Unit (GRU). GRU extensions and variants are named exGRU operators. Parameter items marked (extern) are parameters exclusive to exGRU. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]

        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- linear_before_reset:int

    - The selection of the LBR variant: 1(T) or 0(F)
- direction(extern):string
    - Specifies the computation direction of the GRU
    - forward: specifies the computation direction of the GRU as forward
    - reverse: specifies the computation direction of the GRU as reverse
    - bidirectional: specifies the computation direction of the GRU as bidirectional
- sequence_size(extern):int
    - The seqsize of the GRU input, no limitation, 4-alignment recommended
- hidden_size(extern):int
    - The hiddensize in the GRU cell, no limitation, 8-alignment recommended
- input_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding input shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
    - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
    - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: Not yet supported

Gather

Gathers entries of the specified axis dimension of input X according to indices, and concatenates them together.

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- Indices:T2
  - The indices of the elements to gather, rank = 1

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

Attributes

- axis:int(Tensor)
  - axis∈{0,1,2,3}
  - Specifies the dimension from which Indices obtains the input

Other Support

- Multi-core: Not yet supported

LSTM

Long Short-Term Memory (LSTM). LSTM extensions and variants are named exLSTM operators. Parameter items marked (extern) are parameters exclusive to exLSTM. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]

        - seqlength∈(0,8192]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - inputsize∈(0,8192]
    - Quantization support
        - per-layer
- W:T1
    - shape:[num_directions,4*hidden_size,input_size]
        - inputsize∈(0,8192]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- R:T1
    - shape:[num_directions,4*hidden_size,hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- B:T2,T3
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- sequence_lens:T4,T5(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported

        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- direction(extern):string
  - Specifies the computation direction of the LSTM
  - forward: specifies the computation direction of the LSTM as forward
  - reverse: specifies the computation direction of the LSTM as reverse
  - bidirectional: specifies the computation direction of the LSTM as bidirectional
- sequence_size(extern):int
  - The seqsize of the LSTM input, no limitation, 4-alignment recommended
- hidden_size(extern):int
  - The hiddensize in the LSTM cell, no limitation, 8-alignment recommended
- proj_size(extern):int
  - The proj_size for projection, projsize∈[0,hiddensize]
  - Currently limited to 0, i.e. projection is not yet supported
- input_forget(extern):int
  - cifgvariant selection: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- has_projection(extern):int
  - projection variant: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- input_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding input shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
  - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
  - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding output shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
  - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
  - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)

- T4:float(Scalar)
- T5:int(Scalar)

Other Support

- Multi-core: Not yet supported

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Min

Computes the element-wise minimum value of the input tensor

Inputs

- A:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

MulRelu

The remaining description is the same as Mul

- Supported data types: Same as Mul

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
  - Supported modes: constant, reflect
    - constantNo limitation
    - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]
- storage_order:int64
  - The preferred storage mode: 0

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(1,7]
    - width∈(1,7]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]

    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool consumes an input tensor X and applies average pooling to the tensor according to kernel size, stride size, and padding length. Average pooling consists of computing the average over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)

    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool consumes an input tensor X and applies average pooling over the values in the same channel. This is equivalent to an AveragePool with kernel_shape equal to the spatial dimensions of the input tensor.

Inputs*

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string

    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z = X^Y exponentiation operation, taking input data (tensor) and exponents, and producing one output data (tensor)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- Y:T2
    - shape:[1]
        - Currently only 0, 1, 2, 3, 0.5, -0.5 are supported

Outputs

- Z:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

Other Support

- Multi-core: Not yet supported

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Attributes

- slope: PRelu coefficient
    - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- shape:int
  - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- roi(optional):T2
  - Currently not supported
- scales(optional):T2
  - The array of scale factors along each dimension

- sizes(optional):T3
    - The target size of the output tensor

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)
    - Currently not supported to set
- axes:int[]
    - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")
    - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)
    - Currently not supported to set
- exclude_outside: int (default value 0)
    - Currently not supported to set
- extrapolation_value: float (default 0)
    - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
    - Currently not supported to set
- mode: strings (default value "nearest")
    - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
    - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of 2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- starts:int(Tensor)
  - The start position of the split: No limitation
- ends:int(Tensor)
  - The end position of the split: No limitation
- axes:int(Tensor)
  - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
  - The step size of the corresponding axis for splitting
  - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]

        - Quantization support
            - per-layer
        - Broadcasting support
            - [b,c,1,1]
            - [scalar]

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=8192, q=2048)
  - perm=[0,1,2,3]
    - No limitation
  - perm=[0,1,3,2]
    - Not supported
  - perm=[0,2,3,1]
    - Not supported
  - perm=[0,2,1,3]
    - Not supported
  - perm=[0,3,1,2]
    - Not supported
  - perm=[0,3,2,1]
    - Not supported

- perm=[1,0,2,3]
  - Not supported
- perm=[1,0,3,2]
  - Not supported
- perm=[1,2,0,3]
  - Not supported
- perm=[1,2,3,0]
  - Not supported
- perm=[1,3,0,2]
  - Not supported
- perm=[1,3,2,0]
  - Not supported
- perm=[2,1,0,3]
  - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,h1%4=0)
- perm=[2,1,3,0]
  - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,h1%4=0)
- perm=[2,0,3,1]
  - Not supported
- perm=[2,0,1,3]
  - Not supported
- perm=[2,3,1,0]
  - Not supported
- perm=[2,3,0,1]
  - Not supported
- perm=[3,0,2,1]
  - Not supported
- perm=[3,0,1,2]
  - Not supported
- perm=[3,2,0,1]
  - Not supported
- perm=[3,2,1,0]
  - Not supported
- perm=[3,1,0,2]
  - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,w1%4=0)
- perm=[3,1,2,0]

        - T1,T2:c1<8192,n<8192,h1*w1<8192,(h1=w1=1,n1%4=0)or(n1=h1=1,w1%4=0)

Data Type Constraints

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh, y = 0.5x(1+Tanh(sqrt(2/π)(x+0.044715*x^3)))*

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exSwooshR

Given an input tensor, the following function from the paper https://arxiv.org/abs/2310.11230 is applied element-wise to the input tensor to produce the output tensor SwooshR(x)=log(1+exp(x−1))−0.08x−0.313261687

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exSwooshL

Given an input tensor, the following function from the paper https://arxiv.org/abs/2310.11230 is applied element-wise to the input tensor to produce the output tensor SwooshL(x)=log(1+exp(x−4))−0.08x−0.035

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1

    - shape:[batch,channel,height,width]
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- x_tensor:*T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Output:T1
  - shape:[batch,channel/2,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis: int (default value = 1)
  - The dimension to split the input
  - Constraint: only 1 is supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - Quantization support
    - per-layer
- B:T1
  - shape:[b,k,1,m]
  - Quantization support
    - per-layer

- C(optional):T1,T2
  - shape:[b,n,1,m]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
  - shape:[b,n,1,m]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
  - add and mul are optional, indicating whether C is added or multiplied

---

exNorm

Performs various normalization operations on the input data. Currently three types are supported: LayerNormalization, RMSNormalization, and MeanVarianceNormalization:

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

Where: x is the input vector. μ and σ² are the mean and variance of the input vector respectively:

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$$

RMS is the root mean square of the input vector:

$$\text{RMS} = \sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2}$$

Variance is the equivalent variance of the input vector:

$$\text{Variance} = \frac{1}{H}\sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ is a small number used to prevent division by zero. W and B are optional parameters.

Inputs

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- epsilon: float (default value 1e-05)
  - The epsilon value used to prevent division by 0
- mode:string
  - Three normalization modes are available: layer, rms, and mean_var

- axes: int64[] (default value [1])
    - Normalization axes, currently only [1] and [1,2,3] are supported

Other Support

- Multi-core: Not yet supported

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
    - shape:[batch,channel,height,width]

Outputs

- Y:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- mode:string
    - Supported modes: partition, reverse, partition_num_first
        - partition: implemented with reference to window_partition;
        - reverse: implemented with reference to window_reverse;
        - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of partition are divided according to window_sizes.
- window_sizes:listofints
    - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
    - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported

RK3588

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: Supported

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

Other Support

- Multi-core: Supported

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- epsilon: float (default value = 1e-5)
  - A real number added when dividing by the standard deviation to prevent division by 0
  - epsilon∈(0,∞]
- momentum:float
  - Sliding average parameters during training

Other Support

- Multi-core: Not yet supported

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axisint64
  - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,31]
  - dilations_w∈[1,31]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Supported

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - Quantization support
    - per-layer

        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Supported

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Supported

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Supported

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Supported

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Supported

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Not yet supported

ConvolutionAddRelu

Convolution and Add and Relu fused computation, Output=Relu(Add(Conv(Input0),Input1)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: Supported

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
- Quantization support
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)

    - stride_h only supports 1,2,4,8
    - stride_w only supports 1,2,4,8
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

ConvTranposeRelu

ConvTranspose and Relu fused computation, Output=Relu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeClip

ConvTranspose and Clip fused computation, Output=Clip(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranspose and PRelu/LeakyRelu fused computation, Output=PRelu(ConvTranspose(Input)) or Output= LeakyRelu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeAdd

ConvTranspose and Add fused computation, Output=Add(ConvTranspose(Input0),Input1). Description and specification constraints are the same as ConvTranspose

ConvTranposeSigmoid

ConvTranspose and Sigmoid fused computation, Output=Sigmoid(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeTanh

ConvTranspose and Tanh fused computation, Output=Tanh(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeSoftplus

ConvTranspose and Softplus fused computation, Output=Softplus(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeHardSigmoid

ConvTranspose and HardSigmoid fused computation, Output=HardSigmoid(ConvTranspose(Input))

ConvTranposeHardSwish

ConvTranspose and HardSwish fused computation, Output=HardSwish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeElu

ConvTranspose and Elu fused computation, Output=Elu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeSwish

ConvTranspose and Swish fused computation, Output=Swish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeMish

ConvTranspose and Mish fused computation, Output=Mish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: Not supported

Expand

Broadcast the input tensor according to the given shape and broadcasting rules. The broadcasting rule is similar to numpy.array(input) * numpy.ones(shape): dimensions are right-aligned; two corresponding dimensions must have the same value, or one of them equals 1

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- shape:T2
  - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

Other Support

- Multi-core: Not supported

GRU

Gated Recurrent Unit (GRU). GRU extensions and variants are named exGRU operators. Parameter items marked (extern) are parameters exclusive to exGRU. The computation formulas

are as follows: Equations(Default:f=Sigmoid,g=Tanh): $r_t = f(x_t \cdot W_r + h_{t-1} \cdot R_r + Wb_r + Rb_r)$ $z_t = f(x_t \cdot W_z + h_{t-1} \cdot R_z + Wb_z + Rb_z)$ $h_t = g(x_t \cdot W_h + (r_t \odot h_{t-1}) \cdot R_h + Rb_h + Wb_h)$, (WRB=0) $h_t = g(x_t \cdot W_h + r_t \odot (h_{t-1} \cdot R_h + Rb_h)) + Wb_h$, (WRB=1) $H_t = (1 - z_t) \odot h_t + z_t \odot h_{t-1}$

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- sequence_lens:T3(optional)

    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- linear_before_reset:int
    - The selection of the LBR variant: 1(T) or 0(F)
- direction(extern):string
    - Specifies the computation direction of the GRU
    - forward: specifies the computation direction of the GRU as forward
    - reverse: specifies the computation direction of the GRU as reverse
    - bidirectional: specifies the computation direction of the GRU as bidirectional

- sequence_size(extern):int
  - The seqsize of the GRU input, no limitation, 4-alignment recommended
- hidden_size(extern):int
  - The hiddensize in the GRU cell, no limitation, 8-alignment recommended
- input_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding input shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
  - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
  - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding output shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
  - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
  - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: Not yet supported

Gather

Gathers entries of the specified axis dimension of input X according to indices, and concatenates them together.

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- Indices:T2

    - The indices of the elements to gather, rank = 1

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

Attributes

- axis:int(Tensor)
    - axis∈{0,1,2,3}
    - Specifies the dimension from which Indices obtains the input

Other Support

- Multi-core: Not yet supported

Hardmax

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - axis=1, height∗width<16384;
        - axis=3/-1, channel∗height<16384, and subject to the transpose specification constraints

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - The axis for hardmax: 1,3/-1, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

LSTM

Long Short-Term Memory (LSTM). LSTM extensions and variants are named exLSTM operators. Parameter items marked (extern) are parameters exclusive to exLSTM. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer

        - per-channel
- B:T2,T3
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- sequence_lens:T4,T5(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support

        - per-layer

- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- direction(extern):string
    - Specifies the computation direction of the LSTM
    - forward: specifies the computation direction of the LSTM as forward
    - reverse: specifies the computation direction of the LSTM as reverse
    - bidirectional: specifies the computation direction of the LSTM as bidirectional
- sequence_size(extern):int
    - The seqsize of the LSTM input, no limitation, 4-alignment recommended
- hidden_size(extern):int
    - The hiddensize in the LSTM cell, no limitation, 8-alignment recommended
- proj_size(extern):int
    - The proj_size for projection, projsize∈[0,hiddensize]
    - Currently limited to 0, i.e. projection is not yet supported
- input_forget(extern):int
    - cifgvariant selection: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- has_projection(extern):int
    - projection variant: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- input_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding input shape

    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
    - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
    - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)
- T4:float(Scalar)
- T5:int(Scalar)

Other Support

- Multi-core: Not yet supported

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Min

Computes the element-wise minimum value of the input tensor

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions

        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: Supported

MulRelu

Mul and Relu fused computation, C=Relu(A*B)

Inputs

- Same as Mul

Outputs

- Same as Mul

Other Support

- Multi-core: Supported

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- pads:T2
    - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end, width_end]
- constant_value(optional):T3

        - shape:[1]
        - Quantization support
            - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
    - Supported modes: constant, reflect
        - constantNo limitation
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]

        - width∈(0,8192]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]
- storage_order:int64
    - The preferred storage mode: 0

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,7]
    - width∈(0,7]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool consumes an input tensor X and applies average pooling to the tensor according to kernel size, stride size, and padding length. Average pooling consists of computing the average over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool consumes an input tensor X and applies average pooling over the values in the same channel. This is equivalent to an AveragePool with kernel_shape equal to the spatial dimensions of the input tensor.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,7]
    - width∈(0,7]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]

    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z = X^Y exponentiation operation, taking input data (tensor) and exponents, and producing one output data (tensor)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- Y:T2
    - shape:[1]
        - Currently only 0, 1, 2, 3, 0.5, -0.5 are supported

Outputs

- Z:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

Other Support

- Multi-core: Not yet supported

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Attributes

- slope: PRelu coefficient
    - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- shape:int
    - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- roi(optional):T2
    - Currently not supported
- scales(optional):T2
    - The array of scale factors along each dimension
- sizes(optional):T3
    - The target size of the output tensor

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)
  - Currently not supported to set
- axes:int[]
  - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")
  - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)
  - Currently not supported to set
- exclude_outside: int (default value 0)
  - Currently not supported to set
- extrapolation_value: float (default 0)
  - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
  - Currently not supported to set
- mode: strings (default value "nearest")
  - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
  - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of 2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)
- Multi-core: Not supported

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- starts:int(Tensor)
  - The start position of the split: No limitation
- ends:int(Tensor)
  - The end position of the split: No limitation
- axes:int(Tensor)
  - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
  - The step size of the corresponding axis for splitting
  - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Softmax

This operator computes the normalized exponential value of the given input:

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int
    - The axis for softmax: 1,3, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

---

exSoftmax13

This operator computes the normalized exponential value of the given input:

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - Subject to the transpose specification constraints

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
    - The axis for softmax: 1,3, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

exSoftmaxMask

This operator computes the normalized exponential value of the given input:

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints
- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,No limitation; · axis=3/-1,width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions
- mask_value:int64
  - The values to be masked: 0,1

Other Support

- Multi-core: Not yet supported

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support

        - [b,c,1,1]
        - [scalar]
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - [b,c,1,1]
        - [scalar]

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: Not supported

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Other Support

- Multi-core: Not supported

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=8192, q=2048)
  - perm=[0,1,2,3]
    - No limitation
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q

    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q
- perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
- perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1*w1<p
- perm=[0,3,1,2]
    - T1,T2:h1*w1<p,w1<8192,h1*c1<p
- perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1<16*p,h1*w1<2*q,c1<p*16
    - T2:h1%8=0,h1*w1<8*p,h1*w1<2*q,c1<p*8
- perm=[1,0,2,3]
    - T1:h1*w1<16*p,n1<8192,h1*w1<p
    - T2:h1*w1<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
    - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
    - T1:n1*w1<p*16,c1*h1*w1<p,w1%16=0
    - T2:n1*w1<p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
    - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
    - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
    - T1:c1*w1<8192,n1*h1<p,w1%16=0
    - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
    - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
    - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
    - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
    - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
    - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0

- perm=[2,0,3,1]
  - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,*h1*w1%16=0
  - T2:n1*c1*h1*w1<p,*h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,*h1*w1<8192,h1*w1%16=0
  - T2:c1*h1*w1<p,n1*c1<p,*h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0
  - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%16=0
  - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
  - T1,T2:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]
  - T1:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*16,h1%16=0
  - T2:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*8,h1%8=0
- perm=[3,1,0,2]
  - T1,T2:h1*w1<p,c1*w1<8192,n1*h1<p
- perm=[3,1,2,0]
  - T1,T2:h1*w1<p,n1*c1<8192,n1*h1<p,n1*c1*w1<p

Data Type Constraints

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh, y = 0.5x(1+Tanh(sqrt(2/π)(x+0.044715*x^3)))

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exSwooshR

Given an input tensor, the following function from the paper https://arxiv.org/abs/2310.11230 is applied element-wise to the input tensor to produce the output tensor SwooshR(x)=log(1+exp(x−1))−0.08x−0.313261687

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exSwooshL

Given an input tensor, the following function from the paper https://arxiv.org/abs/2310.11230 is applied element-wise to the input tensor to produce the output tensor SwooshL(x)=log(1+exp(x−4))−0.08x−0.035

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator
- x_tensor:*T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator
- y_tensor:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support

        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exConvStreaming

To reduce the latency of speech models, the input of the original Convolution is divided into several time-frame slices. Each time-frame slice is used as input to compute the convolution, and the results are cached for subsequent time frames, thereby achieving streaming audio processing.

Inputs

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: currently width=1 is supported.
    - Quantization support
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exConvStreamingPRelu/LeakyRelu

exConvStreaming and PRelu/LeakyRelu fused computation, Output = PRelu(exConvStreaming(Input)) or Output= LeakyRelu(exConvStreaming(Input)). Description and specification constraints are the same as exConvStreaming

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1
    - shape:[batch,channel,height,width]

    - Quantization support:per-layer

Outputs

- Output:T1
    - shape:[batch,channel/2,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis: int (default value = 1)
    - The dimension to split the input
    - Constraint: only 1 is supported

Other Support

- Multi-core: Not yet supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
    - shape:[b,k,1,n]
        - k ∈(0,8192]
    - Quantization support
        - per-layer
- B:T1
    - shape:[b,k,1,m]
    - Quantization support
        - per-layer
- C(optional):T1,T2
    - shape:[b,n,1,m]

    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
    - add and mul are optional, indicating whether C is added or multiplied

Other Support

- Multi-core: Not supported

---

exNorm

Performs various normalization operations on the input data. Currently three types are supported: LayerNormalization, RMSNormalization, and MeanVarianceNormalization:

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

Where: x is the input vector. μ and σ² are the mean and variance of the input vector respectively:

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$$

RMS is the root mean square of the input vector:

$$\text{RMS} = \sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2}$$

Variance is the equivalent variance of the input vector:

$$\text{Variance} = \frac{1}{H}\sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ is a small number used to prevent division by zero. W and B are optional parameters.

Inputs

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- epsilon: float (default value 1e-05)
  - The epsilon value used to prevent division by 0
- mode:string
  - Three normalization modes are available: layer, rms, and mean_var

- axes: int64[] (default value [1])
    - Normalization axes, currently only [1] and [1,2,3] are supported

Other Support

- Multi-core: Not yet supported

---

exSDPAttention

Computes scaled dot-product attention (SDPA) between query, key, and value

Inputs

- query:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - Quantization support
        - per-layer
- key:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- value:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - Quantization support
        - per-layer
- mask(optional):T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
- scale(optional):T3

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: Not supported

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- mode:string
  - Supported modes: partition, reverse, partition_num_first
    - partition: implemented with reference to window_partition;
    - reverse: implemented with reference to window_reverse;
    - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of partition are divided according to window_sizes.
- window_sizes:listofints
  - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
  - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported

RK3562

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

Other Support

- Multi-core: Supported

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- epsilon: float (default value = 1e-5)
  - A real number added when dividing by the standard deviation to prevent division by 0
  - epsilon∈(0,∞]
- momentum:float
  - Sliding average parameters during training

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axisint64
    - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
    - Quantization support
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,31]
  - dilations_w∈[1,31]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

---

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAddRelu

Convolution and Add and Relu fused computation, Output=Relu(Add(Conv(Input0),Input1)). Description and specification constraints are the same as Convolution

---

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
- Quantization support
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)

    - stride_h∈[1,8]
    - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

ConvTranposeRelu

ConvTranspose and Relu fused computation, Output=Relu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeClip

ConvTranspose and Clip fused computation, Output=Clip(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranspose and PRelu/LeakyRelu fused computation, Output=PRelu(ConvTranspose(Input)) or Output= LeakyRelu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeAdd

ConvTranspose and Add fused computation, Output=Add(ConvTranspose(Input0),Input1). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSigmoid

ConvTranspose and Sigmoid fused computation, Output=Sigmoid(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeTanh

ConvTranspose and Tanh fused computation, Output=Tanh(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSoftplus

ConvTranspose and Softplus fused computation, Output=Softplus(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranspose and HardSigmoid fused computation, Output=HardSigmoid(ConvTranspose(Input))

ConvTranposeHardSwish

ConvTranspose and HardSwish fused computation, Output=HardSwish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeElu

ConvTranspose and Elu fused computation, Output=Elu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSwish

ConvTranspose and Swish fused computation, Output=Swish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeMish

ConvTranspose and Mish fused computation, Output=Mish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

Inputs

- input1:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

GRU

Gated Recurrent Unit (GRU). GRU extensions and variants are named exGRU operators. Parameter items marked (extern) are parameters exclusive to exGRU. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer

- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- sequence_lens:T3(optional)
  - shape:[batch_size]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - hiddensize∈(0,8192]
  - Quantization support
    - per-layer

Outputs

- Y:T1
  - shape:[seq_length,num_directions,batch_size,hidden_size]
    - seqlength∈(0,8192]
    - numdirections:1or2
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - hiddensize∈(0,8192]
  - Quantization support
    - per-layer
- Y_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - hiddensize∈(0,8192]
  - Quantization support
    - per-layer

Attributes

- linear_before_reset:int
  - The selection of the LBR variant: 1(T) or 0(F)
- direction(extern):string
  - Specifies the computation direction of the GRU
  - forward: specifies the computation direction of the GRU as forward
  - reverse: specifies the computation direction of the GRU as reverse
  - bidirectional: specifies the computation direction of the GRU as bidirectional
- sequence_size(extern):int
  - The seqsize of the GRU input, no limitation, 4-alignment recommended
- hidden_size(extern):int
  - The hiddensize in the GRU cell, no limitation, 8-alignment recommended
- input_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding input shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
  - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
  - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int

    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: Not yet supported

Gather

Gathers entries of the specified axis dimension of input X according to indices, and concatenates them together.

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- Indices:T2
    - The indices of the elements to gather, rank = 1

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

Attributes

- axis:int(Tensor)
  - axis∈{0,1,2,3}
  - Specifies the dimension from which Indices obtains the input

Other Support

- Multi-core: Not yet supported

Hardmax

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - axis=1, height∗width<16384;
    - axis=3/-1, channel∗height<16384, and subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - The axis for hardmax: 1,3/-1, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

LSTM

Long Short-Term Memory (LSTM). LSTM extensions and variants are named exLSTM operators. Parameter items marked (extern) are parameters exclusive to exLSTM. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c +

h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer
- W:T2
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T2
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- sequence_lens:T2(optional)
  - shape:[batch_size]

        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]

        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- direction(extern):string
    - Specifies the computation direction of the LSTM
    - forward: specifies the computation direction of the LSTM as forward
    - reverse: specifies the computation direction of the LSTM as reverse
    - bidirectional: specifies the computation direction of the LSTM as bidirectional
- sequence_size(extern):int
    - The seqsize of the LSTM input, no limitation, 4-alignment recommended
- hidden_size(extern):int
    - The hiddensize in the LSTM cell, no limitation, 8-alignment recommended
- proj_size(extern):int
    - The proj_size for projection, projsize∈[0,hiddensize]
    - Currently limited to 0, i.e. projection is not yet supported
- input_forget(extern):int
    - cifgvariant selection: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- has_projection(extern):int
    - projection variant: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- input_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding input shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
    - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
    - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]

    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)
- T4:float(Scalar)

Other Support

- Multi-core: Not yet supported

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

MulRelu

Mul and Relu fused computation, C=Relu(A*B)

Inputs

- Same as Mul

Outputs

- Same as Mul

Other Support

- Multi-core: Supported

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
    - Supported modes: constant, reflect
        - constantNo limitation
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string

    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]
- storage_order:int64[]
    - The preferred storage mode: 0

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool consumes an input tensor X and applies average pooling to the tensor according to kernel size, stride size, and padding length. Average pooling consists of computing the average over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]

    - width∈(0,8192]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool consumes an input tensor X and applies average pooling over the values in the same channel. This is equivalent to an AveragePool with kernel_shape equal to the spatial dimensions of the input tensor.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,7]
    - width∈(0,7]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z = X^Y exponentiation operation, taking input data (tensor) and exponents, and producing one output data (tensor)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- Y:T2
  - shape:[1]
    - Currently only 0, 1, 2, 3, 0.5, -0.5 are supported

Outputs

- Z:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

Other Support

- Multi-core: Not yet supported

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Attributes

- slope: PRelu coefficient
  - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- shape:int
  - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- roi(optional):T2
  - Currently not supported
- scales(optional):T2
  - The array of scale factors along each dimension
- sizes(optional):T3
  - The target size of the output tensor

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)
  - Currently not supported to set
- axes:int[]
  - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")

    - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)
    - Currently not supported to set
- exclude_outside: int (default value 0)
    - Currently not supported to set
- extrapolation_value: float (default 0)
    - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
    - Currently not supported to set
- mode: strings (default value "nearest")
    - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
    - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of 2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- starts:int(Tensor)

    - The start position of the split: No limitation
- ends:int(Tensor)
    - The end position of the split: No limitation
- axes:int(Tensor)
    - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
    - The step size of the corresponding axis for splitting
    - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Softmax

This operator computes the normalized exponential value of the given input:

`Softmax(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - Subject to the transpose specification constraints

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int
    - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmax13

This operator computes the normalized exponential value of the given input:

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmaxMask

This operator computes the normalized exponential value of the given input:

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,No limitation; · axis=3/-1,width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions
- mask_value:int64
  - The values to be masked: 0,1

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- repeats:T2
  - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=268435456, q=65536)
  - perm=[0,1,2,3]
    - No limitation
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1*<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q,c1<p*16
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1<p*8
  - perm=[1,0,2,3]
    - T1:h1*w1*<16*p,n1<8192,h1*w1<p
    - T2:h1*w1*<8*p,n1<8192,h1*w1<p
  - perm=[1,0,3,2]
    - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
  - perm=[1,2,0,3]
    - T1:n1*w1*<p16,c1*h1*w1<p,w1%16=0
    - T2:n1*w1*<p8,c1*h1*w1p,w1%8=0
  - perm=[1,2,3,0]

    - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
    - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
    - T1:c1*w1<8192,n1*h1<p,w1%16=0
    - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
    - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
    - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
    - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
    - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
    - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
    - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
    - T1:n1*c1*h1*w1<p,h1*w1%16=0
    - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
    - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
    - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
    - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
    - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0
    - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
    - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%16=0
    - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
    - T1,T2:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]
    - T1:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*16,h1%16=0
    - T2:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*8,h1%8=0
- perm=[3,1,0,2]

        - T1,T2:h1*w1<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1<p,n1*c1<8192,n1*h1<p,n1*c1*w1<p

Data Type Constraints

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
    - shape:[batch,channel,height,width]

    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))=x*tanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh, y = 0.5*x(1+Tanh(sqrt(2/π)(x+0.044715*x^3)))

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1
  - shape:[batch,channel,height,width]

    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- x_tensor:*T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Output:T1
  - shape:[batch,channel/2,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis: int (default value = 1)
  - The dimension to split the input
  - Constraint: only 1 is supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - Quantization support
    - per-layer
- B:T1
  - shape:[b,k,1,m]
  - Quantization support
    - per-layer
- C(optional):T1,T2

    - shape:[b,n,1,m]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
    - add and mul are optional, indicating whether C is added or multiplied

---

exNorm

Performs various normalization operations on the input data. Currently three types are supported: LayerNormalization, RMSNormalization, and MeanVarianceNormalization:

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

Where: x is the input vector. μ and σ² are the mean and variance of the input vector respectively:

$$\mu = \frac{1}{H} \sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$$

RMS is the root mean square of the input vector:

$$\mathrm{RMS} = \sqrt{\frac{1}{H}\sum_{i=1}^{H}x_i^2}$$

Variance is the equivalent variance of the input vector:

$$\mathrm{Variance} = \frac{1}{H}\sum_{i=1}^{H}x_i^2 - \mu^2$$

ϵ is a small number used to prevent division by zero. W and B are optional parameters.

Inputs

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- epsilon: float (default value 1e-05)
  - The epsilon value used to prevent division by 0
- mode:string
  - Three normalization modes are available: layer, rms, and mean_var
- axes: int64[] (default value [1])
  - Normalization axes, currently only [1] and [1,2,3] are supported

Other Support

- Multi-core: Not yet supported

---

exSDPAttention

Computes scaled dot-product attention (SDPA) between query, key, and value

Inputs

- query:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - Quantization support
    - per-layer
- key:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- value:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - Quantization support
    - per-layer
- mask(optional):T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - By default, supports broadcasting along arbitrary dimensions
    - After FA support is enabled, only three specifications are supported: [b,c,1,1], [1,c,1,1], and [1,c,h,w]
- scale(optional):T3

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support

    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- FlashAttentionV2 (FA): supported
    - Implemented based on https://arxiv.org/abs/2307.08691. Acceleration is achieved by caching inner loops and reducing bandwidth usage, but this increases the model size. Please choose whether to enable it according to your specific scenario and model
    - RKNPU driver version >= 0.8.8
    - RKNPU Runtime library (librknnrt.so) version >= 2.0.0
    - Supported data type: float16

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- mode:string
  - Supported modes: partition, reverse, partition_num_first
    - partition: implemented with reference to window_partition;
    - reverse: implemented with reference to window_reverse;
    - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of

partition are divided according to window_sizes.

- window_sizes:listofints
  - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
  - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported

RV1103/RV1106

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(constTensor)

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

Other Support

- Multi-core: Supported

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- epsilon: float (default value = 1e-5)
  - A real number added when dividing by the standard deviation to prevent division by 0
  - epsilon∈(0,∞]
- momentum:float
  - Sliding average parameters during training

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axisint64
  - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1

    - shape:[batch,channel,height,width]
        - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
    - Quantization support
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

Data Type Constraints

- T1:int8(Tensor)

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)

- dilations_h∈[1,32]
- dilations_w∈[1,32]

Data Type Constraints

- T1:int8(Tensor)

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

---

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

ConvolutionAddRelu

Convolution and Add and Relu fused computation, Output=Relu(Add(Conv(Input0),Input1)). Description and specification constraints are the same as Convolution

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.

- Quantization support
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,8]
  - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
  - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]
  - When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:int8(Tensor)

ConvTranposeRelu

ConvTranspose and Relu fused computation, Output=Relu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeClip

ConvTranspose and Clip fused computation, Output=Clip(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranspose and PRelu/LeakyRelu fused computation, Output=PRelu(ConvTranspose(Input)) or Output= LeakyRelu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeAdd

ConvTranspose and Add fused computation, Output=Add(ConvTranspose(Input0),Input1). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSigmoid

ConvTranspose and Sigmoid fused computation, Output=Sigmoid(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeTanh

ConvTranspose and Tanh fused computation, Output=Tanh(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSoftplus

ConvTranspose and Softplus fused computation, Output=Softplus(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranspose and HardSigmoid fused computation,
Output=HardSigmoid(ConvTranspose(Input))

ConvTranposeHardSwish

ConvTranspose and HardSwish fused computation, Output=HardSwish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeElu

ConvTranspose and Elu fused computation, Output=Elu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSwish

ConvTranspose and Swish fused computation, Output=Swish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeMish

ConvTranspose and Mish fused computation, Output=Mish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

Broadcast the input tensor according to the given shape and broadcasting rules. The broadcasting rule is similar to numpy.array(input) * numpy.ones(shape): dimensions are right-aligned; two corresponding dimensions must have the same value, or one of them equals 1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- shape:T2
    - shape:[batch,channel,height,width]

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)

- T2:int64(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

Hardmax

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∗height∗width<=8192
    - axis=1, height∗width<16384 and height∗width equal to 1 or greater than 8;
    - axis=3/-1, channel ∗ height < 16384, and channel ∗ height equal to 1 or greater than 8, and subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - The axis for hardmax: 1,3/-1, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

LSTM

Long Short-Term Memory (LSTM). LSTM extensions and variants are named exLSTM operators. Parameter items marked (extern) are parameters exclusive to exLSTM. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i +

h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- sequence_lens:T3(optional)

    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- initial_c:T3(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_c:T1(optional)

    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- direction(extern):string
    - Specifies the computation direction of the LSTM
    - forward: specifies the computation direction of the LSTM as forward
    - reverse: specifies the computation direction of the LSTM as reverse
    - bidirectional: specifies the computation direction of the LSTM as bidirectional
- sequence_size(extern):int
    - The seqsize of the LSTM input, no limitation, 4-alignment recommended
- hidden_size(extern):int
    - The hiddensize in the LSTM cell, no limitation, 8-alignment recommended
- proj_size(extern):int
    - The proj_size for projection, projsize∈[0,hiddensize]
    - Currently limited to 0, i.e. projection is not yet supported
- input_forget(extern):int
    - cifgvariant selection: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- has_projection(extern):int
    - projection variant: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- input_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding input shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
    - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
    - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。

    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:int8(Tensor)
- T2:int32(Tensor)
- T3:int(Scalar)

Other Support

- Multi-core: Not yet supported

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support

        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Min

Computes the element-wise minimum value of the input tensor

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(constTensor)

MulRelu

Mul and Relu fused computation, C=Relu(A*B)

Inputs

- Same as Mul

Outputs

- Same as Mul

Other Support

- Multi-core: Supported

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
    - Supported modes: constant, reflect
        - constantNo limitation
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported

- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]
- storage_order:int64[]
  - The preferred storage mode: 0

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,7]
    - width∈(0,7]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool consumes an input tensor X and applies average pooling to the tensor according to kernel size, stride size, and padding length. Average pooling consists of computing the average over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool consumes an input tensor X and applies average pooling over the values in the same channel. This is equivalent to an AveragePool with kernel_shape equal to the spatial dimensions of the input tensor.

Inputs

- input:T1
  - shape:[batch,channel,height,width]

        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z = X^Y exponentiation operation, taking input data (tensor) and exponents, and producing one output data (tensor)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- Y:T2
  - shape:[1]
    - Currently only 0, 1, 2, 3, 0.5, -0.5 are supported

Outputs

- Z:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

Other Support

- Multi-core: Not yet supported

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Attributes

- slope: PRelu coefficient
  - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- shape:int
  - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- roi(optional):T2
  - Currently not supported
- scales(optional):T2
  - The array of scale factors along each dimension
- sizes(optional):T3
  - The target size of the output tensor

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)
  - Currently not supported to set
- axes:int[]
  - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")
  - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)

    - Currently not supported to set
- exclude_outside: int (default value 0)
    - Currently not supported to set
- extrapolation_value: float (default 0)
    - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
    - Currently not supported to set
- mode: strings (default value "nearest")
    - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
    - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of 2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- starts:int(Tensor)
    - The start position of the split: No limitation
- ends:int(Tensor)

    - The end position of the split: No limitation
- axes:int(Tensor)
    - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
    - The step size of the corresponding axis for splitting
    - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Softmax

This operator computes the normalized exponential value of the given input:

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - Subject to the transpose specification constraints

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int
    - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmax13

This operator computes the normalized exponential value of the given input:

`exSoftmax13(input, axis) = exp(input - max(input, axis)) / Σ_axis exp(input - max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmaxMask

This operator computes the normalized exponential value of the given input:

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints
- mask(optional):T1

- shape:[1,channel,height,width]
  - channel∈(0,8192]
  - width: · axis=1,No limitation; · axis=3/-1,width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions
- mask_value:int64
  - The values to be masked: 0,1

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(constTensor)

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- repeats:T2
  - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1,T2
  - shape:[n2,c2,h2,w2]

        - See Attributes for detailed constraint rules
    - Quantization support
        - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=8192, q=2048)
    - perm=[0,1,2,3]
        - No limitation
    - perm=[0,1,3,2]
        - T1:h1%16=0,h1*w1*<*16*p,h1*w1*<*2*q
        - T2:h1%8=0,h1*w1*<*8*p,h1*w1*<*2*q
    - perm=[0,2,3,1]
        - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
        - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
    - perm=[0,2,1,3]
        - T1:h1%16=0,h1*w1*<*16*p,h1*w1*<*2*q,c1*w1<p
        - T2:h1%8=0,h1*w1*<*8*p,h1*w1*<*2*q,c1*w1<p
    - perm=[0,3,1,2]
        - T1,T2:h1*w1*<*p,w1*<*8192,h1*c1<p
    - perm=[0,3,2,1]
        - T1:h1%16=0,h1*w1*<*16*p,h1*w1*<*2*q,c1<p*16
        - T2:h1%8=0,h1*w1*<*8*p,h1*w1*<*2*q,c1<p*8
    - perm=[1,0,2,3]
        - T1:h1*w1*<*16*p,n1<8192,h1*w1<p
        - T2:h1*w1*<*8*p,n1<8192,h1*w1<p
    - perm=[1,0,3,2]
        - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
        - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
    - perm=[1,2,0,3]
        - T1:n1*w1*<*p*16,c1*h1*w1<p,w1%16=0
        - T2:n1*w1*<*p*8,c1*h1*w1p,w1%8=0
    - perm=[1,2,3,0]
        - T1:h1*w1*<*p,n1*<*p*16,c1*h1*w1<p*16,n1*w1<p*16
        - T2:h1*w1*<*p,n1*<*p*8,c1*h1*w1<p*8,n1*w1<p*8

- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%16=0
  - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
  - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
  - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,h1*w1%16=0
  - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
  - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0
  - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%16=0
  - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
  - T1,T2:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]
  - T1:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*16,h1%16=0
  - T2:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*8,h1%8=0
- perm=[3,1,0,2]
  - T1,T2:h1*w1<p,c1*w1<8192,n1*h1<p
- perm=[3,1,2,0]

        - T1,T2:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

Data Type Constraints

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh, y = 0.5*x*(1+Tanh(sqrt(2/π)*(x+0.044715*x^3)))

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1
    - shape:[batch,channel,height,width]
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions

    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- x_tensor:*T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1

    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Output:T1
    - shape:[batch,channel/2,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis: int (default value = 1)
    - The dimension to split the input
    - Constraint: only 1 is supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
    - shape:[b,k,1,n]
        - k ∈(0,8192]
    - Quantization support
        - per-layer
- B:T1
    - shape:[b,k,1,m]
    - Quantization support
        - per-layer
- C(optional):T1,T2
    - shape:[b,n,1,m]
    - Quantization support

        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
    - add and mul are optional, indicating whether C is added or multiplied

---

exSDPAttention

Computes scaled dot-product attention (SDPA) between query, key, and value

Inputs

- query:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
    - Quantization support
        - per-layer
- key:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- value:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]

        - Quantization support
            - per-layer
- mask(optional):T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - By default, supports broadcasting along arbitrary dimensions
- scale(optional):T3

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- mode:string
  - Supported modes: partition, reverse, partition_num_first
    - partition: implemented with reference to window_partition;
    - reverse: implemented with reference to window_reverse;
    - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of

partition are divided according to window_sizes.

- window_sizes:listofints
  - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
  - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported

RK3576/RV1126B

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- epsilon: float (default value = 1e-5)
  - A real number added when dividing by the standard deviation to prevent division by 0
  - epsilon∈(0,∞]
- momentum:float
  - Sliding average parameters during training

Other Support

- Multi-core: Not yet supported

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axisint64
  - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)

    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
    - Quantization support
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,8]
        - kernel_w∈[1,8]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]

    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvolutionAddRelu

Convolution and Add and Relu fused computation, Output=Relu(Add(Conv(Input0),Input1)). Description and specification constraints are the same as Convolution

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,8]
  - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
  - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)

- dilations_h∈[1,32]
- dilations_w∈[1,32]
- When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

ConvTranposeRelu

ConvTranspose and Relu fused computation, Output=Relu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeClip

ConvTranspose and Clip fused computation, Output=Clip(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranspose and PRelu/LeakyRelu fused computation, Output=PRelu(ConvTranspose(Input)) or Output= LeakyRelu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeAdd

ConvTranspose and Add fused computation, Output=Add(ConvTranspose(Input0),Input1). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSigmoid

ConvTranspose and Sigmoid fused computation, Output=Sigmoid(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeTanh

ConvTranspose and Tanh fused computation, Output=Tanh(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

ConvTranposeSoftplus

ConvTranspose and Softplus fused computation, Output=Softplus(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeHardSigmoid

ConvTranspose and HardSigmoid fused computation, Output=HardSigmoid(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeHardSwish

ConvTranspose and HardSwish fused computation, Output=HardSwish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeElu

ConvTranspose and Elu fused computation, Output=Elu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeSwish

ConvTranspose and Swish fused computation, Output=Swish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeMish

ConvTranspose and Mish fused computation, Output=Mish(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: Not supported

Expand

Broadcast the input tensor according to the given shape and broadcasting rules. The broadcasting rule is similar to numpy.array(input) * numpy.ones(shape): dimensions are right-aligned; two corresponding dimensions must have the same value, or one of them equals 1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- shape:T2

    - shape:[batch,channel,height,width]

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

Other Support

- Multi-core: Not supported

GRU

Gated Recurrent Unit (GRU). GRU extensions and variants are named exGRU operators. Parameter items marked (extern) are parameters exclusive to exGRU. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh): r_t = f(x_t · W_r + h_{t−1} · R_r + Wb_r + Rb_r) z_t = f(x_t · W_z + h_{t−1} · R_z + Wb_z + Rb_z) h_t = g(x_t · W_h + (r_t ⊙ h_{t−1}) · R_h + Rb_h + Wb_h), (WRB=0) h_t = g(x_t · W_h + r_t ⊙ (h_{t−1} · R_h + Rb_h)) + Wb_h, (WRB=1) H_t = (1 − z_t) ⊙ h_t + z_t ⊙ h_{t−1}

Inputs

- X:T1
    - shape:[seq_length,batch_size,input_size]
        - seqlength∈(0,8192]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - inputsize∈(0,8192]
    - Quantization support
        - per-layer

- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- sequence_lens:T3(optional)
  - shape:[batch_size]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - hiddensize∈(0,8192]
  - Quantization support
    - per-layer

Outputs

- Y:T1
  - shape:[seq_length,num_directions,batch_size,hidden_size]
    - seqlength∈(0,8192]
    - numdirections:1or2
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - hiddensize∈(0,8192]
  - Quantization support
    - per-layer
- Y_h:T1(optional)
  - shape:[num_directions,batch_size,hidden_size]
    - numdirections:1or2
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - hiddensize∈(0,8192]
  - Quantization support
    - per-layer

Attributes

- linear_before_reset:int
  - The selection of the LBR variant: 1(T) or 0(F)
- direction(extern):string
  - Specifies the computation direction of the GRU
  - forward: specifies the computation direction of the GRU as forward
  - reverse: specifies the computation direction of the GRU as reverse
  - bidirectional: specifies the computation direction of the GRU as bidirectional
- sequence_size(extern):int
  - The seqsize of the GRU input, no limitation, 4-alignment recommended
- hidden_size(extern):int
  - The hiddensize in the GRU cell, no limitation, 8-alignment recommended
- input_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding input shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
  - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
  - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int

    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: Not yet supported

Gather

Gathers entries of the specified axis dimension of input X according to indices, and concatenates them together.

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- Indices:T2
    - The indices of the elements to gather, rank = 1

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int(Tensor)

Attributes

- axis:int(Tensor)
  - axis∈{0,1,2,3}
  - Specifies the dimension from which Indices obtains the input

Other Support

- Multi-core: Not yet supported

Hardmax

This OP computes the hardmax activation function of the given input, implemented with reference to Hardmax-13 in onnx

$$\text{Hardmax}(input, axis) = \begin{cases} 1 & \text{if } i = \arg\max(input, axis) \\ 0 & \text{otherwise} \end{cases}$$

Inputs

- input:T
  - shape:[batch,channel,height,width]
    - When axis=1, height∗width∈[1,16384);
    - When axis=3/-1, channel∗height∈[1,16384), and subject to the transpose specification constraints

Outputs

- output:T
  - shape:[batch,channel,height,width]

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - The axis for hardmax: axis∈{−1,1,3}, i.e., channel and width directions

---

LSTM

Long Short-Term Memory (LSTM). LSTM extensions and variants are named exLSTM operators. Parameter items marked (extern) are parameters exclusive to exLSTM. The computation

formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

Inputs

- X:T1
  - shape:[seq_length,batch_size,input_size]
    - seqlength∈(0,8192]
    - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
    - inputsize∈(0,8192]
  - Quantization support
    - per-layer
- W:T1
  - shape:[num_directions,4*hidden_size,input_size]
    - inputsize∈(0,8192]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- R:T1
  - shape:[num_directions,4*hidden_size,hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- B:T2,T3
  - shape:[num_directions,8*hidden_size]
    - hiddensize∈(0,8192]
    - numdirections:1or2
  - Quantization support
    - per-layer
    - per-channel
- sequence_lens:T4,T5(optional)

    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_c:T1(optional)

    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- direction(extern):string
    - Specifies the computation direction of the LSTM
    - forward: specifies the computation direction of the LSTM as forward
    - reverse: specifies the computation direction of the LSTM as reverse
    - bidirectional: specifies the computation direction of the LSTM as bidirectional
- sequence_size(extern):int
    - The seqsize of the LSTM input, no limitation, 4-alignment recommended
- hidden_size(extern):int
    - The hiddensize in the LSTM cell, no limitation, 8-alignment recommended
- proj_size(extern):int
    - The proj_size for projection, projsize∈[0,hiddensize]
    - Currently limited to 0, i.e. projection is not yet supported
- input_forget(extern):int
    - cifgvariant selection: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- has_projection(extern):int
    - projection variant: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- input_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding input shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
    - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
    - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
    - Specifies the layout consistent with the meaning of the corresponding output shape
    - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。

    - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
    - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int32(Tensor)
- T4:float(Scalar)
- T5:int(Scalar)

Other Support

- Multi-core: Not yet supported

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Greater

Computes the Greater result of input tensors A and B

Inputs

- A:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Min

Computes the element-wise minimum value of the input tensor

Inputs

- A:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

MulRelu

Mul and Relu fused computation, C=Relu(A*B)

Inputs

- Same as Mul

Outputs

- Same as Mul

Other Support

- Multi-core: RK3576 supported / RV1126B not supported

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
  - Supported modes: constant, reflect
    - constantNo limitation
    - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported

- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]
- storage_order:int64[]
  - The preferred storage mode: 0

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)

    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

AveragePool consumes an input tensor X and applies average pooling to the tensor according to kernel size, stride size, and padding length. Average pooling consists of computing the average over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,8192]
        - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalAveragePool

GlobalAveragePool consumes an input tensor X and applies average pooling over the values in the same channel. This is equivalent to an AveragePool with kernel_shape equal to the spatial dimensions of the input tensor.

Inputs

- input:T1
  - shape:[batch,channel,height,width]

        - channel∈(0,8192]
        - height∈(1,7]
        - width∈(1,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Pow

Z = X^Y exponentiation operation, taking input data (tensor) and exponents, and producing one output data (tensor)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- Y:T2
  - shape:[1]
    - Currently only 0, 1, 2, 3, 0.5, -0.5 are supported

Outputs

- Z:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

Other Support

- Multi-core: Not yet supported

ReduceMax

Obtains the maximum value along the channel direction of the input tensor

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1

    - shape:[batch,1,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer/per-channel

Attributes

- slope: PRelu coefficient
    - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- shape:int
  - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- roi(optional):T2
  - Currently not supported

- scales(optional):T2
  - The array of scale factors along each dimension
- sizes(optional):T3
  - The target size of the output tensor

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)
  - Currently not supported to set
- axes:int[]
  - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")
  - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)
  - Currently not supported to set
- exclude_outside: int (default value 0)
  - Currently not supported to set
- extrapolation_value: float (default 0)
  - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
  - Currently not supported to set
- mode: strings (default value "nearest")
  - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
  - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of

2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)

- Multi-core: Not supported

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- starts:int(Tensor)
  - The start position of the split: No limitation
- ends:int(Tensor)
  - The end position of the split: No limitation
- axes:int(Tensor)
  - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
  - The step size of the corresponding axis for splitting
  - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Softmax

This operator computes the normalized exponential value of the given input:

`Softmax(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

---

exSoftmax13

This operator computes the normalized exponential value of the given input:

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

Other Support

- Multi-core: Not yet supported

---

exSoftmaxMask

This operator computes the normalized exponential value of the given input:

$$\text{input}(x, mask_t, mask_value) = \begin{cases} x + (mask_t - 1) * inf & \text{if } mask_value = 0 \\ x + mask_t * (-inf) & \text{elif } mask_value = 1 \end{cases}$$

$$\text{exSoftmaxMask}(input, axis) = \frac{\exp(input - \max(input, axis))}{\sum_{axis} \exp(input - \max(input, axis))}$$

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints
- mask(optional):T1
  - shape:[1,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,No limitation; · axis=3/-1,width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions
- mask_value:int64
  - The values to be masked: 0,1

Other Support

- Multi-core: Not yet supported

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor),float16(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int

- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(constTensor)

Other Support

- Multi-core: Not supported

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- repeats:T2
  - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Other Support

- Multi-core: Not supported

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=268435456, q=65536)
  - perm=[0,1,2,3]
    - No limitation
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1*<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q,c1<p*16
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1<p*8
  - perm=[1,0,2,3]
    - T1:h1*w1*<16*p,n1<8192,h1*w1<p
    - T2:h1*w1*<8*p,n1<8192,h1*w1<p
  - perm=[1,0,3,2]
    - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
    - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
  - perm=[1,2,0,3]
    - T1:n1*w1*<p16,c1*h1*w1<p,w1%16=0
    - T2:n1*w1*<p8,c1*h1*w1p,w1%8=0
  - perm=[1,2,3,0]

    - T1:h1*w1<p,n1<p*16,c1*h1*w1<p*16,n1*w1<p*16
    - T2:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
    - T1:c1*w1<8192,n1*h1<p,w1%16=0
    - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
    - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
    - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
    - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
    - T1:c1*h1*w1<p,n1*w1<p*16,c1<8192,n1*w1%4=0
    - T2:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
    - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
    - T1:n1*c1*h1*w1<p,h1*w1%16=0
    - T2:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
    - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%16=0
    - T2:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
    - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
    - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%16=0
    - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
    - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%16=0
    - T2:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
    - T1,T2:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]
    - T1:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*16,h1%16=0
    - T2:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*8,h1%8=0
- perm=[3,1,0,2]

        - T1,T2:h1*w1*<p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

Data Type Constraints

- T1:int8(Tensor)
- T2:int16(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))*=xtanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh,

$y = 0.5 * x * (1 + Tanh(\sqrt{\frac{2}{\pi}} * (x + 0.044715 * x^3)))$

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sin

Computes the Sin of the given input tensor element-wise

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

Cos

Computes the Cos of the given input tensor element-wise

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator
- x_tensor:T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator
- y_tensor:T2

    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:int8(Tensor),float16(Tensor)

---

exConvStreaming

To reduce the latency of speech models, the input of the original Convolution is divided into several time-frame slices. Each time-frame slice is used as input to compute the convolution, and the results are cached for subsequent time frames, thereby achieving streaming audio processing.

Inputs

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: currently width=1 is supported.
    - Quantization support
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - Quantization support
        - per-layer

        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exConvStreamingPRelu/LeakyRelu

exConvStreaming and PRelu/LeakyRelu fused computation, Output = PRelu(exConvStreaming(Input)) or Output= LeakyRelu(exConvStreaming(Input)). Description and specification constraints are the same as exConvStreaming

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Output:T1
  - shape:[batch,channel/2,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- axis: int (default value = 1)
  - The dimension to split the input
  - Constraint: only 1 is supported

Other Support

- Multi-core: Not yet supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - Quantization support
    - per-layer
- B:T1
  - shape:[b,k,1,m]
  - Quantization support

        - per-layer

- C(optional):T1,T2
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
    - add and mul are optional, indicating whether C is added or multiplied

Other Support

- Multi-core: Not supported

---

exNorm

Performs various normalization operations on the input data. Currently three types are supported: LayerNormalization, RMSNormalization, and MeanVarianceNormalization:

$$\mathrm{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot W + B$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{RMS}^2 + \epsilon}} \cdot W + B$$

$$\mathrm{MeanVarianceNorm}(x) = \frac{x - \mu}{\sqrt{\mathrm{Variance} + \epsilon}} \cdot W + B$$

Where: x is the input vector. μ and σ² are the mean and variance of the input vector respectively:

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$$

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$$

RMS is the root mean square of the input vector:

$$\mathrm{RMS} = \sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2}$$

Variance is the equivalent variance of the input vector:

$$\mathrm{Variance} = \frac{1}{H}\sum_{i=1}^{H} x_i^2 - \mu^2$$

ϵ is a small number used to prevent division by zero. W and B are optional parameters.

Inputs

- X:T1
  - shape:[batch,channel,height,width]
- W(optional):T1
  - shape:[batch,channel,height,width]
- B(optional):T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- epsilon: float (default value 1e-05)
  - The epsilon value used to prevent division by 0
- mode:string
  - Three normalization modes are available: layer, rms, and mean_var
- axes: int64[] (default value [1])
  - Normalization axes, currently only [1] and [1,2,3] are supported

Other Support

- Multi-core: Not yet supported

---

exSDPAttention

Computes scaled dot-product attention (SDPA) between query, key, and value

Inputs

- query:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - Quantization support
    - per-layer
- key:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- value:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
  - Quantization support
    - per-layer
- mask(optional):T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - By default, supports broadcasting along arbitrary dimensions

        - After FA support is enabled, only three specifications are supported: [b,c,1,1], [1,c,1,1], and [1,c,h,w]

- scale(optional):T3

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: RK3576 supported / RV1126B not supported
- FlashAttentionV2 (FA): supported
    - Implemented based on https://arxiv.org/abs/2307.08691. Acceleration is achieved by caching inner loops and reducing bandwidth usage, but this increases the model size. Please choose whether to enable it according to your specific scenario and model
    - RKNPU driver version >= 0.9.7
    - RKNPU Runtime library (librknnrt.so) version >= 2.1.0
    - Supported data type: float16

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)

Attributes

- mode:string
  - Supported modes: partition, reverse, partition_num_first
    - partition: implemented with reference to window_partition;
    - reverse: implemented with reference to window_reverse;
    - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of

partition are divided according to window_sizes.

- window_sizes:listofints
  - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
  - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported

RK2118

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

Attributes

- epsilon: float (default value = 1e-5)

    - A real number added when dividing by the standard deviation to prevent division by 0
    - epsilon∈(0,∞]
- momentum:float
    - Sliding average parameters during training

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:float16(Tensor)

Attributes

- axisint64
    - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1
    - shape:[batch,channel,height,width]
        - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.

    - Quantization support
        - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,7]
    - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,31]
    - dilations_w∈[1,31]

Data Type Constraints

- T1:float16(Tensor)

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

Data Type Constraints

- T1:float16(Tensor)

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

---

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
- Quantization support
  - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support

        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,8]
    - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:float16(Tensor)

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

Broadcast the input tensor according to the given shape and broadcasting rules. The broadcasting rule is similar to numpy.array(input) * numpy.ones(shape): dimensions are right-aligned; two corresponding dimensions must have the same value, or one of them equals 1

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

- shape:T2
    - shape:[batch,channel,height,width]

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:int64(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

LSTM

Long Short-Term Memory (LSTM). LSTM extensions and variants are named exLSTM operators. Parameter items marked (extern) are parameters exclusive to exLSTM. The computation formulas are as follows: Equations(Default:f=Sigmoid,g=Tanh,h=Tanh): i_t = f(x_t · W_i + h_{t−1} · R_i + Wb_i + Rb_i) f_t = f(x_t · W_f + h_{t−1} · R_f + Wb_f + Rb_f) c_t = f(x_t · W_c + h_{t−1} · R_c + Wb_c + Rb_c) o_t = f(x_t · W_o + h_{t−1} · R_o + Wb_o + Rb_o) C_t = f_t ⊙ C_{t−1} + i_t ⊙ c_t h_t = o_t ⊙ h(C_t)

Inputs

- X:T1
    - shape:[seq_length,batch_size,input_size]
        - seqlength∈(0,8192]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - inputsize∈(0,8192]
    - Quantization support
        - per-layer
- W:T1

    - shape:[num_directions,4*hidden_size,input_size]
        - inputsize∈(0,8192]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- R:T1
    - shape:[num_directions,4*hidden_size,hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- B:T2
    - shape:[num_directions,8*hidden_size]
        - hiddensize∈(0,8192]
        - numdirections:1or2
    - Quantization support
        - per-layer
        - per-channel
- sequence_lens:T3(optional)
    - shape:[batch_size]
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
- initial_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- initial_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2

        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Outputs

- Y:T1
    - shape:[seq_length,num_directions,batch_size,hidden_size]
        - seqlength∈(0,8192]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_h:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer
- Y_c:T1(optional)
    - shape:[num_directions,batch_size,hidden_size]
        - numdirections:1or2
        - batchsize∈(0,8192], when greater than 1, only multiples of 4 are supported
        - hiddensize∈(0,8192]
    - Quantization support
        - per-layer

Attributes

- direction(extern):string
    - Specifies the computation direction of the LSTM
    - forward: specifies the computation direction of the LSTM as forward
    - reverse: specifies the computation direction of the LSTM as reverse
    - bidirectional: specifies the computation direction of the LSTM as bidirectional

- sequence_size(extern):int
  - The seqsize of the LSTM input, no limitation, 4-alignment recommended
- hidden_size(extern):int
  - The hiddensize in the LSTM cell, no limitation, 8-alignment recommended
- proj_size(extern):int
  - The proj_size for projection, projsize∈[0,hiddensize]
  - Currently limited to 0, i.e. projection is not yet supported
- input_forget(extern):int
  - cifgvariant selection: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- has_projection(extern):int
  - projection variant: 1(T) or 0(F), currently limited to 0, i.e. not yet supported
- input_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding input shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size。
  - snc: specifies the input shape corresponding to the layout as [seqs,batches,input_size]
  - (sn)c: specifies the input shape corresponding to the layout as [seqs*batches,input_size,1,1]
- output_layout(extern):int
  - Specifies the layout consistent with the meaning of the corresponding output shape
  - Requires filling in the specified layout, and also requires filling in the batch_size, sequence_size, hid- den_size、directions。
  - sbnc: specifies the output shape corresponding to the layout as [seqs,directions,batches,hidden_size]
  - (sn)c: specifies the output shape corresponding to the layout as [seqs*batches,directions*input_size,1,1]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

Other Support

- Multi-core: Not yet supported

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]

        - Quantization support
            - per-layer
        - Broadcasting support
            - Supports broadcasting along arbitrary dimensions
            - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

MulRelu

Mul and Relu fused computation, C=Relu(A*B)

Inputs

- Same as Mul

Outputs

- Same as Mul

Other Support

- Multi-core: Supported

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
  - Supported modes: constant, reflect
    - constantNo limitation
    - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - height∈(0,8192]
    - width∈(0,8192]

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
  - kernel_h∈(0,7]
  - kernel_w∈(0,7]
  - Note: kernel_h=1 and kernel_w=1 are not supported
- auto_pad:string
  - Padding mode: only NOTSET is supported
- ceil_mode:int64
  - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]

    - pad_right∈[0,7]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7] Note: kernel_h=1 and kernel_w=1 are not supported
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]
    - count_include_pad, whether to include pad values in computation: 1

- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

- Not supported (converted to convolution implementation on the tool side)

GlobalAveragePool

- Not supported (converted to convolution implementation on the tool side)

Pow

Z = X^Y exponentiation operation, taking input data (tensor) and exponents, and producing one output data (tensor)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- Y:T2
  - shape:[1]
    - Currently only 0, 1, 2, 3, 0.5, -0.5 are supported

Outputs

- Z:T1
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:int8(Tensor),int32(Tensor),int64(Tensor),float16(Tensor),float(Tensor)

Other Support

- Multi-core: Not yet supported

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Broadcasting support
        - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Attributes

- slope: PRelu coefficient
  - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)

Attributes

- shape:int
    - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- roi(optional):T2
    - Currently not supported
- scales(optional):T2
    - The array of scale factors along each dimension
- sizes(optional):T3
    - The target size of the output tensor

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)

    - Currently not supported to set
- axes:int[]
    - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")
    - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)
    - Currently not supported to set
- exclude_outside: int (default value 0)
    - Currently not supported to set
- extrapolation_value: float (default 0)
    - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
    - Currently not supported to set
- mode: strings (default value "nearest")
    - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
    - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of 2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:float16(Tensor)

Attributes

- starts:int(Tensor)
  - The start position of the split: No limitation
- ends:int(Tensor)
  - The end position of the split: No limitation
- axes:int(Tensor)
  - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
  - The step size of the corresponding axis for splitting
  - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Softmax

This operator computes the normalized exponential value of the given input:

`Softmax(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,No limitation; · axis=3/-1,width∈(0,8192] and subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmax13

This operator computes the normalized exponential value of the given input:

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - channel∈(0,8192]
    - width: · axis=1,No limitation; · axis=3/-1,width∈(0,8192] and subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmaxMask

- Not yet supported

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:float16(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]

        - Quantization support
            - per-layer
        - Broadcasting support
            - [b,c,1,1]
            - [scalar]

Outputs

- C:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
- repeats:T2
    - shape:[batch,channel,height,width]

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1
  - shape:[n2,c2,h2,w2]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=268435456, q=65536)
  - perm=[0,1,2,3]
    - No limitation
  - perm=[0,1,3,2]
    - T1:h1%8=0,h1*w1*<*8*p,h1*w1*<*2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%8=0,h1*w1*<*8*p,h1*w1*<*2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1:h1*w1*<*p,w1*<*8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%8=0,h1*w1*<*8*p,h1*w1*<*2*q,c1<p*8

- perm=[1,0,2,3]
  - T1:h1*w1<8*p,n1<8192,h1*w1<p
- perm=[1,0,3,2]
  - T1:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
  - T1:n1*w1<p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
  - T1:h1*w1<p,n1<p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1<p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1<p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1<p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
  - T1:h1*w1<p,n1*c1<8192,c1*h1<p,n1*w1<8192,n1*h1*w1%8=0
- perm=[3,2,0,1]
  - T1:h1*w1<p,n1*c1<8192,n1*c1*w1<p
- perm=[3,2,1,0]
  - T1:c1*h1*w1<p,h1*w1<8192,2*h1*w1<q,n1*c1<p*8,h1%8=0
- perm=[3,1,0,2]
  - T1:h1*w1<p,c1*w1<8192,n1*h1<p
- perm=[3,1,2,0]

        - T1:h1*w1*<p,n1*c1<8192,n1*h1*<p,n1*c1*w1<p

Data Type Constraints

- T1:int16(Tensor),float16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Outputs

- Y:T1
    - shape:[batch,channel,height,width]
    - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]

    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh, y = 0.5*x(1+Tanh(sqrt(2/π)(x+0.044715*x^3)))*

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:float16(Tensor)

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1
    - shape:[batch,channel,height,width]
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions

        - Broadcasting constraints
            - Broadcasting constraints are the same as the Expand operator
- x_tensor:*T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator
- y_tensor:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints
        - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:float16(Tensor)

---

exConvStreaming

To reduce the latency of speech models, the input of the original Convolution is divided into several time-frame slices. Each time-frame slice is used as input to compute the convolution, and the results are cached for subsequent time frames, thereby achieving streaming audio processing.

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: currently width=1 is supported.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,32]
  - dilations_w∈[1,32]

Data Type Constraints

- T1:float16(Tensor)

Other Support

- Multi-core: Not yet supported

---

exConvStreamingPRelu/LeakyRelu

exConvStreaming and PRelu/LeakyRelu fused computation, Output = PRelu(exConvStreaming(Input)) or Output= LeakyRelu(exConvStreaming(Input)). Description and specification constraints are the same as exConvStreaming

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Output:T1
  - shape:[batch,channel/2,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:float16(Tensor)

Attributes

- axis: int (default value = 1)
  - The dimension to split the input
  - Constraint: only 1 is supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - Quantization support
    - per-layer
- B:T1
  - shape:[b,k,1,m]
  - Quantization support
    - per-layer
- C(optional):T1,T2
  - shape:[b,n,1,m]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
  - shape:[b,n,1,m]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
  - add and mul are optional, indicating whether C is added or multiplied

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:float16(Tensor)

Attributes

- mode:string

    - Supported modes: partition, reverse, partition_num_first
        - partition: implemented with reference to window_partition;
        - reverse: implemented with reference to window_reverse;
        - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of partition are divided according to window_sizes.
- window_sizes:listofints
    - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
    - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported

RV1103B/RV1106B

Add

Performs element-wise binary addition(C=A+B),supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(constTensor)

AddRelu

Add and Relu fused computation, C=Relu(A+B)

Inputs

- Same as Add

Outputs

- Same as Add

Other Support

- Multi-core: Supported

BatchNormalization

Performs batch normalization on the input tensor according to the description in https://arxiv.org/abs/1502.03167

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support

        - per-layer

Data Type Constraints

- T1:int8(Tensor)

Attributes

- epsilon: float (default value = 1e-5)
    - A real number added when dividing by the standard deviation to prevent division by 0
    - epsilon∈(0,∞]
- momentum:float
    - Sliding average parameters during training

Concat

Concatenate a sequence of vectors into a single vector along the axis specified by axis. All vectors must be the same size except in the axis dimension

Inputs

- input_tensor:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:int8(Tensor)

Attributes

- axisint64
    - axis∈{0,1,2,3}

Convolution

Convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,31]
    - kernel_w∈[1,31]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]
  - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
  - dilations_h∈[1,31]
  - dilations_w∈[1,31]

Data Type Constraints

- T1:int8(Tensor)

Depthwise Convolution

Depthwise separable convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
  - Quantization support
    - per-layer
- weight:T1
  - shape:[output_channel,input_channel,kernel_h,kernel_w]
    - kernel_h∈[1,8]
    - kernel_w∈[1,8]
  - Quantization support
    - per-layer
    - per-channel

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Attributes

- strides:int(strides_h,strides_w)
  - stride_h∈[1,7]
  - stride_w∈[1,7]
- pads:int(pads_top,pads_left,pads_bottom,pads_right)
  - pads_top∈[0,15]
  - pads_left∈[0,15]
  - pads_bottom∈[0,15]

    - pads_right∈[0,15]
- group:int
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]

Data Type Constraints

- T1:int8(Tensor)

ConvolutionRelu

Convolution and Relu fused computation, Output=Relu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionClip

Convolution and Clip fused computation, Output=Clip(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionPRelu/LeakyRelu

Convolution and PRelu/LeakyRelu fused computation, Output = PRelu(Conv(Input)) or Output = LeakyRelu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionAdd

Convolution and Add fused computation, Output=Add(Conv(Input0),Input1). Description and specification constraints are the same as Convolution

---

ConvolutionSigmoid

Convolution and Sigmoid fused computation, Output=Sigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionTanh

Convolution and Tanh fused computation, Output=Tanh(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSoftplus

Convolution and Softplus fused computation, Output=Softplus(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSigmoid

Convolution and HardSigmoid fused computation, Output=HardSigmoid(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionHardSwish

Convolution and HardSwish fused computation, Output=HardSwish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionElu

Convolution and Elu fused computation, Output=Elu(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionSwish

Convolution and Swish fused computation, Output=Swish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvolutionMish

Convolution and Mish fused computation, Output=Mish(Conv(Input)). Description and specification constraints are the same as Convolution

---

ConvTranspose

Transposed convolution

Inputs

- input_tensor:T1
  - shape:[batch,channel,height,width]
    - width: when dilated_kernel_h>1, width<16383. In addition, the first-layer input width is restricted; see the model input description for details.
- Quantization support

    - per-layer
- weight:T1
    - shape:[output_channel,input_channel,kernel_h,kernel_w]
        - kernel_h∈[1,31]
        - kernel_w∈[1,31]
    - Quantization support
        - per-layer
        - per-channel

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Attributes

- strides:int(strides_h,strides_w)
    - stride_h∈[1,8]
    - stride_w∈[1,8]
- pads:int(pads_top,pads_left,pads_bottom,pads_right) Note when setting pad: kernel_h*dilations_h-dilations_h-pads_top<0 is not supported; kernel_w*dilations_w-dilations_w-pads_left<0 is not supported; stride_h*(height-1)-pads_top+1<output_h is not supported; stride_w*(width-1)-pads_left+1<output_w is not supported
    - pads_top∈[0,15]
    - pads_left∈[0,15]
    - pads_bottom∈[0,15]
    - pads_right∈[0,15]
- group:int
    - group: only 1 is supported; num_output is supported if and only if num_input=num_output
- dilations:int(dilations_h,dilations_w)
    - dilations_h∈[1,32]
    - dilations_w∈[1,32]
    - When dilation is not equal to (1,1), only kernel=(1,1) is supported

Data Type Constraints

- T1:int8(Tensor)

ConvTranposeRelu

ConvTranspose and Relu fused computation, Output=Relu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposeClip

ConvTranspose and Clip fused computation, Output=Clip(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

ConvTranposePRelu/LeakyRelu

ConvTranspose and PRelu/LeakyRelu fused computation, Output=PRelu(ConvTranspose(Input)) or Output= LeakyRelu(ConvTranspose(Input)). Description and specification constraints are the same as ConvTranspose

---

Div

Performs element-wise binary division (C=A/B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:float16(Tensor)
- T2:float(constTensor)

Expand

Broadcast the input tensor according to the given shape and broadcasting rules. The broadcasting rule is similar to numpy.array(input) * numpy.ones(shape): dimensions are right-aligned; two corresponding dimensions must have the same value, or one of them equals 1

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- shape:T2
  - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:int64(Tensor)

Broadcasting constraints

- [b,c,1,w]->[b,c,h,w],h∈[1,8192]
- [b,1,1,w]->[b,c,h,w],h∈[1,8192]
- [b,c,h,1]->[b,c,h,w],w ∈[1,8192]

- [b,1,h,1]->[b,c,h,w],w ∈[1,8192]
- [b,c,1,1]->[b,c,h,w],h∈[1,8192],w ∈[1,8192]

Max

Computes the element-wise maximum value of the input tensor

Inputs

- A:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Data Type Constraints

- T1:int8(Tensor)

Other Support

- Multi-core: Not yet supported

Mul

Performs element-wise binary multiplication (C=A*B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
    - Broadcasting constraints are the same as the Expand operator

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(constTensor)

MulRelu

Mul and Relu fused computation, C=Relu(A*B)

Inputs

- Same as Mul

Outputs

- Same as Mul

Other Support

- Multi-core: Supported

Pad

Given a tensor (data) containing the data to be padded, a tensor (pads) containing the start and end padding values for each axis, a mode (mode), and a constant value (constant_value), generate a padded tensor (output)

Inputs

- data:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- pads:T2
  - shape : [batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end,height_end,width_end]
- constant_value(optional):T3
  - shape:[1]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:int64(Tensor)
- T3:float(Scalar),int8(Scalar),float16(Scalar)

Attributes

- mode: string (default value constant)
  - Supported modes: constant, reflect

        - constantNo limitation
        - reflectchannel∈(0,8192],height∈(0,8192],width∈(0,8176]

Other Support

- Multi-core: Not yet supported

MaxPool

MaxPool consumes an input tensor X and applies max pooling to the tensor according to kernel size, stride size, and padding length. Max pooling consists of computing the maximum over all values of a subset of the input tensor according to the kernel size, and downsampling the data into output tensor Y for further processing. The output spatial shape calculation differs depending on whether explicit padding (when using pads) or automatic padding (when using auto_pad) is used. Only explicit padding is used

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported
- strides:int64(strides_h,strides_w)
    - strides_h∈(0,8]
    - strides_w∈(0,8]
- count_include_pad:int64[]

    - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
    - pad_top∈[0,7]
    - pad_left∈[0,7]
    - pad_bottom∈[0,7]
    - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

GlobalMaxPool

GlobalMaxPool consumes an input tensor X and applies max pooling over the values in the same channel. This is equivalent to a MaxPool with kernel_shape equal to the spatial dimensions of the input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - channel∈(0,8192]
        - height∈(0,7]
        - width∈(0,7]

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Attributes

- kernel_shape:int64(kernel_h,kernel_w)
    - kernel_h∈(0,7]
    - kernel_w∈(0,7]
- auto_pad:string
    - Padding mode: only NOTSET is supported
- ceil_mode:int64
    - Whether to use ceil or floor to compute the output shape: not supported

- strides:int64(strides_h,strides_w)
  - strides_h∈(0,8]
  - strides_w∈(0,8]
- count_include_pad:int64[]
  - count_include_pad, whether to include pad values in computation: 1
- pad:int64(pad_top,pad_left,pad_bottom,pad_right)
  - pad_top∈[0,7]
  - pad_left∈[0,7]
  - pad_bottom∈[0,7]
  - pad_right∈[0,7]

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:float(Scalar)

AveragePool

- Not supported (converted to convolution implementation on the tool side)

GlobalAveragePool

- Not supported (converted to convolution implementation on the tool side)

Relu

Sets negative values of the input tensor to zero, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)

Other Support

- Multi-core: Not yet supported

Clip/ReLU6

Clips the values of the input tensor to the range [0,6]

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer
    - Broadcasting support
        - None

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)

Other Support

- Multi-core: Not yet supported

PRelu

Scales negative values of the input tensor by a learnable parameter, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer/per-channel

Attributes

- slope: PRelu coefficient
  - Only a single scalar or C-dimension coefficients are supported

Data Type Constraints

- T1:int8(Tensor)

Other Support

- Multi-core: Not yet supported

LeakyRelu

Scales negative values of the input tensor by a fixed ratio, keeping positive values unchanged

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - None

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

Other Support

- Multi-core: Not yet supported

Reshape

Returns a tensor with the specified shape without changing the input data or the number of elements.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

Attributes

- shape:int
  - Specifies the shape of the output tensor

Other Support

- Multi-core: Not yet supported

Resize

Resizes the input tensor. In general, it computes each value in the output tensor as a weighted average of the input tensor values in the neighborhood (i.e., sampling locations) of the corresponding position

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- roi(optional):T2
  - Currently not supported
- scales(optional):T2
  - The array of scale factors along each dimension
- sizes(optional):T3
  - The target size of the output tensor

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)
- T3:int64(Tensor)

Attributes

- antialias: int (default value 0)
  - Currently not supported to set
- axes:int[]
  - Currently not supported to set
- coordinate_transformation_mode: strings (default value "half_pixel")

    - Currently only half_pixel, pytorch_half_pixel, and align_corners are supported
- cubic_coeff_a: float (default -0.75)
    - Currently not supported to set
- exclude_outside: int (default value 0)
    - Currently not supported to set
- extrapolation_value: float (default 0)
    - Currently not supported to set
- keep_aspect_ratio_policy: strings (default value "stretch")
    - Currently not supported to set
- mode: strings (default value "nearest")
    - Currently only nearest and linear are configurable
- nearest_mode: strings (default value "round_prefer_floor")
    - Currently not supported to set Support Note Currently only integer scaling in width/height direction is supported, and the scale factor must be an integer multiple of 2-8. If the input is (1,c,1,1), it can be scaled up to (1,c,h,w)

Slice

Obtains a slice of the current vector, with the same rules as numpy slicing

Inputs

- input_tensor:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T:int8(Tensor)

Attributes

- starts:int(Tensor)

    - The start position of the split: No limitation
- ends:int(Tensor)
    - The end position of the split: No limitation
- axes:int(Tensor)
    - The axis to split: any axis 0~3 is supported, and multiple axes can be selected simultaneously
- steps:int(Tensor)
    - The step size of the corresponding axis for splitting
    - When height*width==1&&channel%subc==0&&starts[1]%subc==0&&ends[1]% when subc==0&&steps[1]<=subc, steps can be other than 1; otherwise, Slice OP can only run on the NPU when steps==[1,1,1,1]

Softmax

This operator computes the normalized exponential value of the given input:

`Softmax(input, axis) = exp(input – max(input, axis)) / Σ_axis exp(input – max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
    - shape:[batch,channel,height,width]
        - Subject to the transpose specification constraints

Outputs

- output:T1
    - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor)

Attributes

- axis:int64
    - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmax13

This operator computes the normalized exponential value of the given input:

`exSoftmax13(input, axis) = exp(input − max(input, axis)) / Σ_axis exp(input − max(input, axis))`

The "axis" attribute indicates the dimension along which Softmax is performed. The output tensor has the same shape and contains the Softmax values of the corresponding input.

Inputs

- input:T1
  - shape:[batch,channel,height,width]
    - Subject to the transpose specification constraints

Outputs

- output:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor)

Attributes

- axis:int64
  - The axis for softmax: 1,3, i.e., channel and width directions

---

exSoftmaxMask

- Not yet supported

Split

Splits the vector into num_outputs vectors along the axis direction. split specifies the size of each split vector along the axis

Inputs

- input_tensor:T
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T
  - One or more output vectors, determined by the num_outputs attribute
  - shape: determined by the splitting situation
  - Quantization support
    - per-layer

Data Type Constraints

- T:int8(Tensor)

Attributes

- axis:int
  - axis∈{0,1,2,3}
- num_outputs:int
- split:int(Tensor)

Sub

- Performs element-wise binary subtraction (C=A-B), supporting multi-directional NumPy-style broadcasting

Inputs

- A:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]
- B:T1,T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - [b,c,1,1]
    - [scalar]

Outputs

- C:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(constTensor)

Tile

Constructs a tensor by tiling a given tensor. This is the same as the tile function in Numpy, but without broadcasting

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
- repeats:T2
  - shape:[batch,channel,height,width]

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor),float16(Tensor)
- T2:int64(Tensor)

Transpose

Transposes the input tensor

Inputs

- X:T1
  - shape:[n1,c1,h1,w1]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Outputs

- Y:T1,T2
  - shape:[n2,c2,h2,w2]
    - See Attributes for detailed constraint rules
  - Quantization support
    - per-layer

Attributes

- The axis order of the perm transpose: The perm parameter restrictions are as follows: (where p=268435456, q=65536)
  - perm=[0,1,2,3]
    - No limitation
  - perm=[0,1,3,2]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q
  - perm=[0,2,3,1]
    - T1:c1<8192,h1*w1%8=0，w1*c1<p*16
    - T2:c1<8192,h1*w1%8=0，w1*c1<p*8
  - perm=[0,2,1,3]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q,c1*w1<p
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1*w1<p
  - perm=[0,3,1,2]
    - T1,T2:h1*w1*<p,w1<8192,h1*c1<p
  - perm=[0,3,2,1]
    - T1:h1%16=0,h1*w1*<16*p,h1*w1*<2*q,c1<p*16
    - T2:h1%8=0,h1*w1*<8*p,h1*w1*<2*q,c1<p*8
  - perm=[1,0,2,3]
    - T1:h1*w1*<16*p,n1<8192,h1*w1<p
    - T2:h1*w1*<8*p,n1<8192,h1*w1<p

- perm=[1,0,3,2]
  - T1:h1%16=0,c1*h1*w1<16*p,h1*w1<2*q
  - T2:h1%8=0,c1*h1*w1<8*p,h1*w1<2*q
- perm=[1,2,0,3]
  - T1:n1*w1*<*p*16,c1*h1*w1<p,w1%16=0
  - T2:n1*w1*<*p*8,c1*h1*w1p,w1%8=0
- perm=[1,2,3,0]
  - T1:h1*w1*<*p,n1*<*p*16,c1*h1*w1<p*16,n1*w1<p*16
  - T2:h1*w1*<*p,n1*<*p*8,c1*h1*w1<p*8,n1*w1<p*8
- perm=[1,3,0,2]
  - T1:c1*w1<8192,n1*h1<p,w1%16=0
  - T2:c1*w1<8192,n1*h1<p,w1%8=0
- perm=[1,3,2,0]
  - T1:c1*h1*w1<p,n1*h1<p,w1<8192,w1%16=0
  - T2:c1*h1*w1<p,n1*h1<p,w1<8192,w1%8=0
- perm=[2,1,0,3]
  - T1,T2:c1<8192,c1*h1*w1<p,n1*w1<p
- perm=[2,1,3,0]
  - T1:c1*h1*w1<p,n1*w1*<*p*16,c1<8192,n1*w1%4=0
  - T2:c1*h1*w1<p,n1*w1*<*p*8,c1<8192,n1*w1%4=0
- perm=[2,0,3,1]
  - T1,T2:c1<8192,c1*h1*w1<p
- perm=[2,0,1,3]
  - T1:n1*c1*h1*w1*<*p,h1*w1%16=0
  - T2:n1*c1*h1*w1*<*p,h1*w1%8=0
- perm=[2,3,1,0]
  - T1:c1*h1*w1<p,n1*c1*<*p,h1*w1<8192,h1*w1%16=0
  - T2:c1*h1*w1<p,n1*c1*<*p,h1*w1<8192,h1*w1%8=0
- perm=[2,3,0,1]
  - T1,T2:h1*w1<8192,n1*c1<p
- perm=[3,0,2,1]
  - T1:h1*w1*<*p,n1*c1<8192,c1*h1*<*p,n1*w1<8192,n1*w1%16=0
  - T2:h1*w1*<*p,n1*c1<8192,c1*h1*<*p,n1*w1<8192,n1*w1%8=0
- perm=[3,0,1,2]
  - T1:h1*w1*<*p,n1*c1<8192,c1*h1*<*p,n1*w1<8192,n1*h1*w1%16=0

        - T2:h1*w1*<*p,n1*c1<8192,c1*h1*<*p,n1*w1<8192,n1*h1*w1%8=0
    - perm=[3,2,0,1]
        - T1,T2:h1*w1*<*p,n1*c1<8192,n1*c1*w1<p
    - perm=[3,2,1,0]
        - T1:c1*h1*w1<p,h1*w1*<*8192,2*h1*w1*<*q,n1*c1<p*16,h1%16=0
        - T2:c1*h1*w1<p,h1*w1*<*8192,2*h1*w1*<*q,n1*c1<p*8,h1%8=0
    - perm=[3,1,0,2]
        - T1,T2:h1*w1*<*p,c1*w1<8192,n1*h1<p
    - perm=[3,1,2,0]
        - T1,T2:h1*w1*<*p,n1*c1<8192,n1*h1*<*p,n1*c1*w1<p

Data Type Constraints

- T1:int8(Tensor)
- T2:int16(Tensor)

Other Support

- Multi-core: Not yet supported

Sigmoid

Given an input tensor, the function y=1/(1+exp(-x)) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Outputs

- output:T1
    - shape:[batch,channel,height,width]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)

Tanh

Computes the hyperbolic tangent of the given input tensor element-wise

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor)

Softplus

Applies the Softplus function element-wise, y=ln(exp(x)+1)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor)

HardSigmoid

Given an input tensor, the function y=max(0,min(1,alpha*x+beta)) is applied element-wise to the input tensor to produce the output tensor, where alpha=1/6, beta=0.5

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

HardSwish

Applies the HardSwish function element-wise, y = x * max(0, min(1, alpha * x + beta)) = x * HardSigmoid(x), where alpha=1/6, beta=0.5

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor)

Elu

Given an input tensor, the function f(x)=alpha*(exp(x)-1.) for x<0, f(x)=x for x>=0 is applied element-wise to the input tensor to produce the output tensor, where alpha=1

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

---

exSwish

Applies the Sigmoid Linear Unit function element-wise. The Swish function is also known as the SiLU function, y=x*sigmoid(x)

Inputs

- X:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Y:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor)

---

exMish

Given an input tensor, the function mish(x)=x*tanh(softplus(x))*=x*tanh(ln(1+e^{x})) is applied element-wise to the input tensor to produce the output tensor

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

---

exGelu

Given an input tensor, the function y = x*Φ(x) is applied element-wise to the input tensor to produce the output tensor. When Φ(x) is set to tanh,

$y = 0.5 * x * (1 + Tanh(\sqrt{\frac{2}{\pi}} * (x + 0.044715 * x^3)))$

Inputs

- input:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Outputs

- output:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:int8(Tensor)

Where

Obtains values from x_tensor or y_tensor according to mask_tensor. When the position of mask_tensor is True, the value of x_tensor at the corresponding position is taken; otherwise, the value of y_tensor is taken.

Inputs

- mask_tensor:T1
  - shape:[batch,channel,height,width]
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator
- x_tensor:*T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator
- y_tensor:T1
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer
  - Broadcasting support
    - Supports broadcasting along arbitrary dimensions
  - Broadcasting constraints
    - Broadcasting constraints are the same as the Expand operator

Outputs

- output:T2
  - shape:[batch,channel,height,width]
  - Quantization support
    - per-layer

Data Type Constraints

- T1:bool(Tensor)
- T2:int8(Tensor)

---

exGlu

Gated Linear Unit function, where a is the first half of the input matrix and b is the second half GLU(a,b)=a⊗σ(b)

Inputs

- Input:T1
  - shape:[batch,channel,height,width]
  - Quantization support:per-layer

Outputs

- Output:T1
  - shape:[batch,channel/2,height,width]
  - Quantization support:per-layer

Data Type Constraints

- T1:int8(Tensor)

Attributes

- axis: int (default value = 1)
  - The dimension to split the input
  - Constraint: only 1 is supported

---

exMatMul

Matrix multiplication operation

$$Y = \begin{cases} A \cdot B + C & \text{if } c_type = \text{"add"} \\ A \cdot B * C & \text{if } c_type = \text{"mul"} \end{cases}$$

Inputs

- A:T1
  - shape:[b,k,1,n]
    - k ∈(0,8192]
  - Quantization support

        - per-layer
- B:T1
    - shape:[b,k,1,m]
    - Quantization support
        - per-layer
- C(optional):T1,T2
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer
    - Broadcasting support
        - Supports broadcasting along arbitrary dimensions
        - Broadcasting constraints are the same as the Expand operator

Outputs

- Y:T1
    - shape:[b,n,1,m]
    - Quantization support
        - per-layer

Data Type Constraints

- T1:int8(Tensor)
- T2:float(Tensor)

Attributes

- c_type: strings (default "add")
    - add and mul are optional, indicating whether C is added or multiplied

---

exWindow

exWindow originates from the window_partition and window_reverse operations of swin_transformer. In the original paper, input features are divided into multiple non-overlapping windows, each of size window_size × window_size. Both input and output features are required to be 4-dimensional shapes, partitioned in both height and width directions. The pytorch implementation is as follows:

```
def window_partition(x, window_size):
    """
    Args:
        x: (B, C, H, W)
        window_size (int): window size
    Returns:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
    """
    B, C, H, W = x.shape
    x = x.view(B, C, H // window_size, window_size, W // window_size, window_size)
    windows = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, -1, window_size *
window_size)
    return windows

def window_reverse(windows, window_size, H, W):
    """
    Args:
        windows: (B, C, num_windows * num_windows, window_size * window_size)
        window_size (int): Window size
        H (int): Height of image
        W (int): Width of image
    Returns:
        x: (B, C, H, W)
    """
    B, C, _, _ = windows.shape
    x = windows.view(B, C, H // window_size, W // window_size, window_size,
window_size)
    x = x.permute(0, 1, 2, 4, 3, 5).contiguous().view(B, C, H, W)
    return x
```

Inputs

- X:T1
  - shape:[batch,channel,height,width]

Outputs

- Y:T1
  - shape:[batch,channel,height,width]

Data Type Constraints

- T1:int8(Tensor)

Attributes

- mode:string
  - Supported modes: partition, reverse, partition_num_first
    - partition: implemented with reference to window_partition;
    - reverse: implemented with reference to window_reverse;
    - partition_num_first: the difference from partition is that the H,W of partition_num_first are divided according to num_windows, while the H,W of

partition are divided according to window_sizes.

- window_sizes:listofints
  - A 2-dimensional vector, the window size. Defines the height and width of each window. Attention computation and feature extraction within the window are based on this size.
- num_windows:listofints
  - A 2-dimensional vector, the number of windows. Defines how many windows the height and width of the input feature are divided into.

Other Support

- Multi-core: Not yet supported
- 3566/3568 OP specification introduction
- 3588 OP specification introduction
- 3562 OP specification introduction
- 1103/1106 OP specification introduction
- 3576/RV1126B OP specification introduction
- 2118 OP specification introduction
- 1103B/1106B OP specification introduction

CPU OP Specifications

Note:

- RK2118 does not support all CPU operators
- RV1106/RV1103/RV1106B/RV1103B only support some CPU operators as follows
    - Concat, DataConvert, Div, exLayerNorm, Exp, Gather, ReduceL2, ReduceMax, Reshape, ScatterND,Softmax,Pow

---

Add

- **Description**: Addition operation
- **Specification Constraints**: No limitation

AveragePool

- **Description**: Average pooling
- **Specification Constraints**: No limitation

ArgMin

- **Description**: Returns the index of the minimum value
- **Specification Constraints**: No limitation

ArgMax

- **Description**: Returns the index of the maximum value
- **Specification Constraints**: No limitation

BatchNormalization

- **Description**: Batch normalization
- **Specification Constraints**: No limitation

Cast

- **Description**: Data type conversion

- **Specification Constraints**: SRC supports:float32/bool/int8/float16/int32/int64 DST supports:float32/int8/int32/float16

Clip

- **Description**: Clipping activation layer
- **Specification Constraints**: No limitation

Concat

- **Description**: Concatenation operation
- **Specification Constraints**: axis only supports {0,1,2,3}

Convolution

- **Description**: Convolution operation
- **Specification Constraints**: No limitation

ConvTranspose/Deconvolution

- **Description**: Transposed convolution
- **Specification Constraints**: No limitation

ConvTransposePad

- **Description**: Combined operator
- **Specification Constraints**: No limitation

Cos

- **Description**: Cosine function
- **Specification Constraints**: No limitation

DataConvert

- **Description**: Data type conversion
- **Specification Constraints**: only bool/int8/float type conversion is supported

---

DepthToSpace

- **Description**: Channel-to-spatial direction conversion
- **Specification Constraints**: No limitation

Div

- **Description**: Division operation
- **Specification Constraints**: No limitation

Equal

- **Description**: Equal to
- **Specification Constraints**: No limitation

Exp

- **Description**: Exponential function
- **Specification Constraints**: No limitation

Flatten

- **Description**: Flatten operation
- **Specification Constraints**: No limitation

Gather

- **Description**: Gather operation
- **Specification Constraints**: No limitation

Greater

- **Description**: Greater than
- **Specification Constraints**: No limitation

GreaterOrEqual

- **Description**: Greater than or equal to
- **Specification Constraints**: No limitation

GRU

- **Description**: Gated Recurrent Unit
- **Specification Constraints**: No limitation

---

exGRU

- **Description**: Gated Recurrent Unit
- **Specification Constraints**: No limitation
    - Note: ONNX extension operator

---

exHardSwish

- **Description**: Activation function
- **Specification Constraints**: No limitation
    - Note: ONNX extension operator

InstanceNormalization

- **Description**: Instance normalization
- **Specification Constraints**: No limitation

---

exLayerNorm

- **Description**: Layer normalization
- **Specification Constraints**: No limitation
    - Note: ONNX extension operator

Less

- **Description**: Less than
- **Specification Constraints**: No limitation

LessOrEqual

- **Description**: Less than or equal to
- **Specification Constraints**: No limitation

LogSoftmax

- **Description**: Activation function
- **Specification Constraints**: batchsize only supports 1

LpNormalization

- **Description**: Lp normalization
- **Specification Constraints**: No limitation

---

exLRN

- **Description**: Local response normalization
- **Specification Constraints**: No limitation
    - Note: ONNX extension operator

MatMul

- **Description**: Multi-dimensional matrix multiplication
- **Specification Constraints**: No limitation (4D x 4D and 4D x 3D computation are supported)

Max

- **Description**: Maximum
- **Specification Constraints**: No limitation

MaxPool

- **Description**: Max pooling
- **Specification Constraints**: No limitation

MaxRoiPool

- **Description**: Region Max pooling
- **Specification Constraints**: No limitation

MaxUnpool

- **Description**: Reverse Max pooling
- **Specification Constraints**: No limitation

---

exMish

- **Description**: Activation function
- **Specification Constraints**: No limitation
    - Note: ONNX extension operator

Min

- **Description**: Minimum value
- **Specification Constraints**: No limitation

Mul

- **Description**: Multiplication
- **Specification Constraints**: No limitation

Pad

- **Description**: Padding
- **Specification Constraints**: No limitation

Pow

- **Description**: Exponentiation
- **Specification Constraints**: No limitation

---

exProposal

- **Description**: Region proposal network
- **Specification Constraints**: batchsize only supports 1
    - Note: ONNX extension operator

ReduceMax

- **Description**: Compute Max along the specified dimension
- **Specification Constraints**: the output dimension cannot exceed 4 dimensions

---

ReduceMean

- **Description**: Compute Mean along the specified dimension
- **Specification Constraints**: the output dimension cannot exceed 4 dimensions

---

ReduceSum

- **Description**: Compute Sum along the specified dimension
- **Specification Constraints**: the output dimension cannot exceed 4 dimensions

---

ReduceMin

- **Description**: Compute Min along the specified dimension
- **Specification Constraints**: the output dimension cannot exceed 4 dimensions

---

Reorg

- **Description**: Data reorganization
- **Specification Constraints**: No limitation

Reshape

- **Description**: Data shape change
- **Specification Constraints**: No limitation

Resize

- **Description**: Data width/height direction scaling
- **Specification Constraints**: supported interpolation methods are bilinear; nearest2d

---

ReverseSequence

- **Description**: Sequence reversal
- **Specification Constraints**: No limitation

exRMSNorm

- **Description**: Root mean square normalization
- **Specification Constraints**: No limitation
    - Note: ONNX extension operator

RoiAlign

- **Description**: Region align pooling
- **Specification Constraints**: only AvgPoolMode is supported, batchsize only supports 1

ScatterND

- **Description**: N-dimensional index gathering
- **Specification Constraints**: No limitation

Sin

- **Description**: Sine function
- **Specification Constraints**: No limitation

Slice

- **Description**: Slice operation
- **Specification Constraints**: batchsize only supports 1

Softmax

- **Description**: Activation function
- **Specification Constraints**: batchsize only supports 1
    - Note: consistent with the ONNX OPSET11 specification

---

exSoftmax13

- **Description**: Activation function
- **Specification Constraints**: batchsize only supports 1
    - Note: ONNX extension operator, consistent with the ONNX OPSET13 specification

SpaceToDetph

- **Description**: Spatial-to-channel direction conversion
- **Specification Constraints**: No limitation

Split

- **Description**: Split data
- **Specification Constraints**: No limitation

Sqrt

- **Description**: Square root
- **Specification Constraints**: No limitation

Squeeze

- **Description**: Squeeze data dimensions
- **Specification Constraints**: No limitation

Sub

- **Description**: Subtraction
- **Specification Constraints**: No limitation

Tanh

- **Description**: Hyperbolic tangent function
- **Specification Constraints**: No limitation

Tile

- **Description**: Expand and copy data
- **Specification Constraints**: batchsize only supports 1, broadcasting is not supported

---

Transpose

- **Description**: Transpose computation

- **Specification Constraints**: No limitation

Upsample

- **Description**: Upsample
- **Specification Constraints**: supported interpolation methods are bilinear; nearest2d

---

Not

- **Description**: Element-wise negation
- **Specification Constraints**: No limitation

---

where

- **Description**: Gather by mask
- **Specification Constraints**: No limitation

Erf

- **Description**: Error function (Erf)
- **Specification Constraints**: No limitation

Floor

- **Description**: Floor operation function
- **Specification Constraints**: No limitation

Mod

- **Description**: Modulo function
- **Specification Constraints**: No limitation

---

exMeanVarianceNormalization

- **Description**: Mean variance normalization function
- **Specification Constraints**: No limitation

And

- **Description**: AND function
- **Specification Constraints**: No limitation

GatherElements

- **Description**: Element gathering function
- **Specification Constraints**: No limitation

Log

- **Description**: Log function
- **Specification Constraints**: No limitation

---

exDataConvert

- **Description**: Precision conversion
- **Specification Constraints**: No limitation

SpaceToDepth

- **Description**: Dimension transformation
- **Specification Constraints**: No limitation

SoftmaxMask

- **Description**: where-fused softmax activation function
- **Specification Constraints**: No limitation

TransposeReshape

- **Description**: Combined operator
- **Specification Constraints**: No limitation

ReshapeTranspose

- **Description**: Combined operator
- **Specification Constraints**: No limitation

ReduceL2

- **Description**: L2 norm along an axis
- **Specification Constraints**: No limitation

LayerNormalization

- **Description**: Layer normalization
- **Specification Constraints**: No limitation

PassThrough

- **Description**: Direct connection
- **Specification Constraints**: No limitation

Expand

- **Description**: Tensor data copy or expansion
- **Specification Constraints**: No limitation

Sigmoid

- **Description**: Activation function
- **Specification Constraints**: No limitation

Tanh

- **Description**: Trigonometric functions
- **Specification Constraints**: No limitation

Cos

- **Description**: Trigonometric functions
- **Specification Constraints**: No limitation

Sin

- **Description**: Trigonometric functions
- **Specification Constraints**: No limitation

exSwish

- **Description**: Activation function
- **Specification Constraints**: No limitation

---

exGlu

- **Description**: Activation function
- **Specification Constraints**: No limitation

---

exGelu

- **Description**: Activation function
- **Specification Constraints**: No limitation

---

exWindow

- **Description**: exWindow originates from the window_partition and window_reverse operations of Swin-Transformer
- **Specification Constraints**: No limitation

---

exNorm

- **Description**: When mode is set to 'pixel', exNorm will use PixelNorm for normalization. When mode is omitted, exNorm uses LayerNorm for normalization by default.
- **Specification Constraints**: No limitation

Round

- **Description**: Data saturation
- **Specification Constraints**: No limitation

TopK

- **Description**: Get the top K ranked values
- **Specification Constraints**: No limitation

---

exSwooshR

- **Description**: Activation function
- **Specification Constraints**: No limitation

---

exSwooshL

- **Description**: Activation function
- **Specification Constraints**: No limitation

OneHot

- **Description**: One-hot encoding
- **Specification Constraints**: No limitation

Cumsum

- **Description**: Cumulative sum along the specified dimension
- **Specification Constraints**: No limitation

GPU OP Specifications

Note:

- RV1106/RV1103/RV1106B/RV1103B/RV1126B/RK2118 do not support GPU operators
- RK356X does not yet support GridSample, exGelu

GridSample

- **Description**: Interpolation sampling
- **Specification Constraints**: only FP16 is supported, the interpolation algorithm only supports bilinear, padding_mode only supports zeros

---

GRU

- **Description**: Gated Recurrent Unit
- **Specification Constraints**: only the case where hidden_size is 4-aligned is supported; the case where input_size and hidden_size are less than 4 is not supported

---

exGelu

- **Description**: Activation function
- **Specification Constraints**: Int8 implementation is currently supported

---

Model I/O Limitations

Model Input Limitations

1. The alignment constraints apply only to the zero-copy API; the normal API has no such constraints.
2. The input width alignment requirement can be queried from the w_stride attribute of the zero-copy API. Note: w_stride cannot be changed.
3. Only the input width has alignment requirements under different channel conditions; there are no other constraints.
4. If mean and scale are not needed for the input, set mean to 0 and scale to 1.
5. If channel > 4, mean/scale uniformly use the first value, i.e., mean[0] and scale[0].
6. If the first layer is floating-point type, there is no quant operation.
7. RV1106/RV1103/RV1103B do not support CPU mean/scale/quant operations.
8. For detailed usage, refer to "Rockchip_RKNPU_User_Guide_RKNN_SDK".

RK3566/RK3568

<table>
<tr><th rowspan="2">芯片
平台</th><th rowspan="2">模型首层
精度类型</th><th rowspan="2">输入
维度</th><th rowspan="2">首层设置输
入数据类型</th><th rowspan="2">mean/scale/quant
后端实现设备</th><th colspan="2">输入宽（width）对齐要求单
位：元素个数</th><th colspan="2">输入宽（width）大小限制</th></tr>
<tr><th>当输入通道
（channel）
为 1,3,4</th><th>当输入通道
（channel）
非 1,3,4</th><th>当输入通道（channel）为 1,3,4</th><th>当输入通道
（channel）
非 1,3,4</th></tr>
<tr><td rowspan="6">RK3566
/3568</td><td rowspan="4">int8</td><td rowspan="5">4 维度</td><td>uint8</td><td rowspan="2">NPU</td><td rowspan="4">8</td><td rowspan="4">1</td><td rowspan="4">各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式：
1. width * dilation_kernel_h < 1024*N
2. 2. width <= 4096
其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下：
Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)
Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)</td><td rowspan="4">无限制</td></tr>
<tr><td>int8</td></tr>
<tr><td>float16</td><td rowspan="2">CPU</td></tr>
<tr><td>其他类型
输入对齐要
求可能变动</td></tr>
<tr><td rowspan="4">float16</td><td>uint8</td><td rowspan="4">CPU</td><td rowspan="4">4</td><td rowspan="4">1</td><td rowspan="4">各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式：
1. width * dilation_kernel_h < 1024*N
2. 2. width <= 4096
其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下：
Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)
Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 128)</td><td rowspan="4">无限制</td></tr>
<tr><td>int8</td></tr>
<tr><td>float16</td></tr>
<tr><td>其他类型
输入对齐要
求可能变动</td></tr>
<tr><td></td><td>无限制</td><td>非 4 维</td><td>非限制</td><td>CPU</td><td>1</td><td>1</td><td>无限制</td><td>无限制</td></tr>
</table>

RK3588

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK3588	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 8192 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3) Depthwise Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 2048), 3) ConvTranspose/Deconvolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3)	无限制
			int8					
			float16	CPU				
			其他类型 输入对齐要求可能变动					
	float16		uint8	CPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 1024 * N 2. width <= 8192 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3) Depthwise Convolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 2048), 3) ConvTranspose/Deconvolution: N = 12 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 3)	无限制
			int8					
			float16					
			其他类型 输入对齐要求可能变动					
	无限制	非 4 维	非限制	CPU	1	1	无限制	无限制

RV1103/RV1106

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4（声明见注释 9）	当输入通道（channel）非 1,3,4
RV1103/1106	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制

RK3562

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK3562	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					
			float16	CPU				
			其他类型输入对齐要求可能变动					
	float16		uint8	CPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					
			float16					
			其他类型输入对齐要求可能变动					
	无限制	非 4 维	非限制	CPU	1	1	无限制	无限制

RK3576/RV1126B

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK3576 / RV1126B	int8	4 维度	uint8	NPU	16	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation kernel w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 2048) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation kernel h * dilation kernel w) / 128), 2)	无限制
			int8					
			float16	CPU				
			其他类型输入对齐要求可能变动					
	float16		uint8	CPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation kernel w) / 128), 2) Depthwise Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 2048), 3) ConvTranspose/Deconvolution: N = 12 - MAX(CEIL((dilation kernel h * dilation kernel w) / 128), 2)	无限制
			int8					
			float16					
			其他类型输入对齐要求可能变动					
	无限制	非 4 维	非限制	CPU	1	1	无限制	无限制

RK2118

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4
RK2118	float16	4 维度	float16	CPU	4	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 3. width * dilation_kernel_h <= 2048 * N 4. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
		非 4 维			1	1		

RV1103B

芯片平台	模型首层精度类型	输入维度	首层设置输入数据类型	mean/scale/quant后端实现设备	输入宽（width）对齐要求单位：元素个数		输入宽（width）大小限制	
					当输入通道（channel）为 1,3,4	当输入通道（channel）非 1,3,4	当输入通道（channel）为 1,3,4（声明见注释 9）	当输入通道（channel）非 1,3,4
RV1103B/RV1106B	int8	4 维度	uint8	NPU	8	1	各卷积类型的 width/kernel_h/kernel_w 需要满足以下两式： 1. width * dilation_kernel_h <= 2048 * N 2. width <= 4096 其中 N 必须为 1 到 7 的整数，超出范围的卷积不受支持，各卷积类型 N 的计算方式如下： Convolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2) Depthwise Convolution: N = 8 - CEIL((dilation_kernel_h * dilation_kernel_w) / 4096) ConvTranspose/Deconvolution: N = 8 - MAX(CEIL((dilation_kernel_h * dilation_kernel_w) / 128), 2)	无限制
			int8					

Model Output Limitations

1. If the output tensor type is NHWC and the output conversion is implemented by the NPU, there are alignment requirements; CPU implementations have no alignment requirements.
2. The output precision type int8/float16 indicates the data type of the raw output of the last layer of the model.
3. For NCHW output, if implemented by the NPU with the zero-copy interface, the output memory size should be based on the queried size.
4. For NC1HWC2 output, the output memory size should be based on the queried size.

RK3566/RK3568

芯片平台	模型输出精度类型	输出维度	设置输出 Layout	Channel 对齐要求	H*W 对齐要求
RK3566/RK3568	int8	4 维度	NCHW	无	无
			NHWC	8 对齐	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	float16		NCHW	无	无
			NHWC	4 对齐	
			NC1HWC2	最后一层卷积类算子，8 对齐，最后一层非卷积类算子 4 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	无限制	非 4 维度	UNDEFINE	无	无

RK3588

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK3588	int8	4 维度	NCHW	无	无
			NHWC	16 对齐	无
			NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	float16		NCHW	无	无
			NHWC	8 对齐	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	无限制	非 4 维度	UNDEFINE	无	无

RV1103/RV1106

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RV1103 /RV1106	int8	4 维度	NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			NHWC	无	无

RK3562

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK3562	int8	4 维度	NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	float16		NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDEFINE	无	无
	无限制	非 4 维度	UNDEFINE	无	无

RK3576/RV1126B

芯片平台	模型输出精度类型	输出维度	设置输出Layout	Channel 对齐要求	H*W 对齐要求
RK3576/ RV1126B	int8	4 维度	NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，32 对齐，最后一层非卷积类算子 16 对齐	H*W 要 4 对齐
			UNDE-FINE	无	无
	float16		NCHW	无	无
			NHWC	无	无
			NC1HWC2	最后一层卷积类算子，16 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			UNDE-FINE	无	无
	无限制	非 4 维度	UNDE-FINE	无	无

RK2118

芯片平台	模型输出精度类型	输出维度	设置输出 Layout	Channel 对齐要求	H*W 对齐要求
RK2118	float16	4 维度	NC1HWC2	最后一层卷积类算子，8 对齐，最后一层非卷积类算子 4 对齐	H*W 要 4 对齐
			NHWC	无	无
		非 4 维度	UNDEFINE	无	无

RV1103B/RV1106B

芯片平台	模型输出精度类型	输出维度	设置输出 Layout	Channel 对齐要求	H*W 对齐要求
RV1103B/ RV1106B	int8	4 维度	NC1HWC2	最后一层卷积类算子，8 对齐，最后一层非卷积类算子 8 对齐	H*W 要 4 对齐
			NHWC	无	无

Special Notes

dilated_kernel_h

`dilated_kernel_h = kernel_h * dilations_h - dilations_h + 1`

dilated_kernel_w

`dilated_kernel_w = kernel_w * dilations_w - dilations_w + 1`